

Ain Shams University

Faculty of Computer and Information Sciences

Software Engineering Program



Questify



June 2026



Ain Shams University
Faculty of Computer & Information Sciences
Software Engineering Department

Questify

**This documentation is submitted in partial fulfillment of the requirements for the
Bachelor's degree in Computer and Information Sciences**

By

Faris Osama Mohamed [SWE]

Omar Waleed AbdElMonaem [SWE]

Mahmoud Mohamed Abdelglil [SWE]

Pierre Atef Farag [SWE]

Ali Ahmed Shawky [SWE]

Nader George Sobhy [SWE]

Under Supervision of

Dr. Salsabil Amin

Lecturer,

Basic Sciences Department,

Faculty of Computer and Information Sciences, Ain
Shams University.

TA. Manar Sultan

Teaching Assistant,

Scientific Computing Department,

Faculty of Computer and Information Sciences, Ain
Shams University.

June 2026

Acknowledgement

All praise and thanks to ALLAH, who provided us the ability to complete this work. We hope to accept this work from us.

We are grateful to our parents and our families who are always providing help and support throughout the whole years of study. We hope we can give that back to them.

We also offer our sincerest gratitude to our supervisors, Dr. Salsabil Amin and T.A Manar Sultan who have supported us throughout our thesis with their patience, knowledge and experience.

Finally, we would like to thank our friends and all the people who gave us support and encouragement.

Abstract

The rapid growth of mobile learning has intensified demand for platforms that sustain long-term engagement while accommodating diverse, instructor-authored curricula. Although gamification and large-language-model (LLM)-based tutoring have each been studied extensively, systems that integrate both into a single, end-to-end, type-safe pipeline — from content ingestion through personalized quest delivery to real-time progress feedback — remain rare. This paper presents Questify, a full-stack gamified learning platform engineered as a single TypeScript monorepo comprising three deployable applications (an Express RESTful API, an Expo React Native mobile client, and a React administrator dashboard) bound together by a shared Zod schema package that guarantees structural and validation consistency across the entire stack. Questify's design is organized around a multi-stage quest model (informational → multiple-choice → boss-battle), a layered gamification engine (XP, levels, streaks with freeze mechanics, badges across five families and four tiers, achievements, and daily/weekly challenges), and an AI augmentation pipeline that ingests instructor-uploaded course documents, generates vector embeddings via the pgvector PostgreSQL extension, and exposes retrieval-augmented question answering through a dedicated "course-ask" module backed by a durable pg-boss job queue. We describe the system's architecture, its data model of thirty Prisma entities, the design rationale for unifying AI-generated content, mastery tracking, and reward delivery behind a single progression contract, and the engineering practices — shared types, background workers for asynchronous generation, and accessibility-aware reward overlays — that together allow the platform to scale from a single course to a multi-tenant deployment without compromising type safety or learner experience.

Table of Contents

Questify.....	2
Abstract.....	2
Table of Contents.....	3
List of Figures.....	6
List of Abbreviations.....	7
1.1 Motivation.....	2
1.2 Problem Definition.....	2
1.3 Objective.....	3
1.4 Document Organization.....	4
Chapter Two:.....	6
Background.....	6
2.1 The Mobile Learning Landscape.....	7
2.2 Gamification in Education.....	7
2.3 Habit Formation and Streak Mechanics.....	8
2.4 AI-Augmented Personalized Learning.....	8
2.5 Retrieval-Augmented Generation in Education.....	8
2.6 Existing Similar Systems.....	9
3.1 System Overview.....	11
3.1.1 System Architecture.....	11
3.1.2 Functional Requirements.....	12
3.1.3 Nonfunctional Requirements.....	20
3.1.4 System Users.....	22
3.2 System Analysis & Design.....	26
3.2.1 Use Case Diagram.....	26
3.2.2 Class Diagram.....	30
Core Entities.....	31
Gamification & Economy.....	32
Observability & Content Pipeline.....	32
3.2.3 Sequence Diagrams.....	33
3.2.4 Database Diagram.....	34
4.1 Questify's Purpose.....	39
4.2 System Functions.....	39
4.2.1 Course Content Management.....	39
4.2.2 User Progression.....	39

4.2.3 Stage Mastery Tracking.....	40
4.2.4 Challenges.....	40
4.2.5 Achievements.....	41
4.2.6 Badges.....	41
4.2.7 Leaderboards.....	41
4.2.8 Shop and Inventory.....	41
4.2.9 Streak Freeze.....	42
4.2.10 RAG-Powered Chatbot.....	42
4.2.11 AI Content Generation.....	42
4.2.12 Notifications.....	43
4.2.13 Authentication and Authorization.....	43
4.2.14 Audit Logging.....	43
4.2.15 Analytics and Reporting.....	44
4.3 Techniques and Algorithms.....	44
4.3.1 Event-Driven Architecture.....	44
4.3.2 Stage Mastery Algorithm.....	44
4.3.3 Leveling Algorithm.....	45
4.3.4 Retrieval-Augmented Generation Pipeline.....	45
4.3.5 Streak Freeze Algorithm.....	45
4.3.6 Idempotent Achievement Evaluation.....	45
4.3.7 Leaderboard Snapshot Pattern.....	46
4.3.8 Content Ordering Service.....	46
4.3.9 Job Queue Processing.....	46
4.4 Technologies Used.....	47
4.4.1 Expo and React Native.....	47
4.4.2 Express and Prisma.....	47
4.4.3 PostgreSQL with pgvector.....	47
4.4.4 pg-boss Job Queue.....	47
4.4.5 AI SDK and Language Models.....	47
4.4.6 Local Embeddings with Transformers.js.....	48
4.4.7 LangChain Text Splitters.....	48
4.4.8 TanStack Query and Zustand.....	48
4.4.9 Radix UI and Recharts.....	48
4.4.10 Zod Schema Validation.....	48
4.4.11 Supporting Technologies.....	49

Chapter Five:.....	50
System Manual.....	50
5.1 Operation Manual.....	51
Mobile App:.....	51
Admin Dashboard:.....	62
5.2 Installation Guide.....	63
Prerequisites.....	63
Install.....	63
Environment (minimum required to run).....	64
Database setup.....	65
Run.....	65
Mobile development build.....	66
First run.....	66
6.1 Conclusions.....	68
6.2 Evaluation Overview.....	68
6.2.1 Overall Content Quality.....	68
6.2.2 Results by Course.....	69
6.2.3 Results by Stage Type.....	69
6.2.4 Observed Weaknesses.....	69
6.2.5 Summary of Key Findings.....	70
6.3 Future Work.....	70
References.....	73

List of Figures

Figure	Caption	Page
Fig. 1.1	System Architecture.....	11
Fig. 3.1	Use Case Diagram— Administrative Management	26
Fig. 3.2	Use Case Diagram— Authentication & Account.....	27
Fig. 3.3	Use Case Diagram— Gamification & Rewards.....	28
Fig. 3.4	Use Case Diagram— Student Learning Journey.....	29
Fig. 3.5	Class Diagram.....	30
Fig. 3.6	Sequence Diagram.....	33
Fig. 3.7	Database Diagram.....	34

List of Abbreviations

API	Application Programming Interface
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MCQ	Multiple-Choice Question
MOOC	Massive Open Online Course
ORM	Object-Relational Mapping
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SDK	Software Development Kit
URL	Uniform Resource Locator
XP	Experience Points

Chapter One:

Introduction

1.1 Motivation

Mobile learning has grown rapidly, with smartphones now a primary mode of self-directed study. Yet two challenges continue to limit its impact. First, learner engagement decays sharply after the first few sessions; completion rates for self-paced courses frequently fall below 15% [1]. Gamification — the use of game mechanics such as points, badges, streaks, and progression in non-game contexts — improves short-term engagement [3], but most deployed implementations are shallow, with rewards bolted onto traditional e-learning systems as cosmetic flourishes rather than woven into the educational core.

Second, large language models (LLMs) and retrieval-augmented generation (RAG) have opened a new design space for personalized tutoring [2], yet integration remains immature. Most AI tutors operate as isolated chat interfaces with no awareness of a learner's enrolled course, progress, or assessment outcomes, producing a fragmented experience in which content delivery, assessment, gamified feedback, and AI assistance exist as separate, loosely coupled tools.

This work is therefore motivated by a central question: can engagement, AI augmentation, and instructor-led content authoring be unified into a single coherent learning system?

1.2 Problem Definition

Despite the proliferation of mobile learning platforms, learners disengage quickly, AI tutoring remains detached from course material, and instructors lack efficient ways to turn raw teaching content into structured, engaging

learning experiences. Existing systems typically address these concerns in isolation: gamification layers are bolted on as cosmetic rewards with no real pedagogical escalation, AI assistants answer questions in a curriculum-agnostic vacuum, and content authors must rely on specialized effort to convert notes and slides into scaffolded activities. The result is a fragmented learner experience in which progression, assessment, and personal assistance have no meaningful connection to one another, and the feedback loop between teaching, practice, and AI help is broken. Questify is built specifically to close this gap by unifying gamified progression, retrieval-grounded AI assistance, and instructor-led content authoring into a single end-to-end platform.

1.3 Objective

The general objective of this project is to design, implement, and document a unified, AI-augmented gamified learning platform that sustains learner engagement, supports instructor-led content authoring, and demonstrates an end-to-end pipeline from document ingestion to in-app personalized assistance. Specifically, the project aims to introduce a multi-stage quest model (informational $\rightarrow$ multiple-choice $\rightarrow$ boss-battle) that structures learning as escalating challenge, and to back it with a layered gamification engine comprising XP, levels, streaks with freeze mechanics, badges, achievements, and time-bound challenges; the platform further integrates retrieval-augmented generation as a first-class learning feature, so that instructor-uploaded documents are embedded and used to ground a "course-ask" interface against the learner's currently enrolled course. Beyond the learner experience, the project delivers a coherent instructor workflow through a web dashboard for authoring, course management, and progress

review, connected by a shared content and progress model so that instructor changes appear immediately for learners.

1.4 Document Organization

This document is organized into the following chapters:

- **Chapter 2** provides the background context for the project, exploring the mobile learning landscape, the role of gamification in education, and the psychology behind habit formation. It also reviews recent advancements in AI-augmented learning and retrieval-augmented generation (RAG), culminating in a survey of existing educational platforms to highlight the gap that Questify fills.
- **Chapter 3** details the analysis and design of the platform, outlining the system architecture, functional and non-functional requirements, and the intended user groups. It includes key design artifacts such as use case diagrams, class diagrams, sequence diagrams, and the database schema that underpin the system's logic and data integrity.
- **Chapter 4** covers the implementation phase, describing the system's modules, technical algorithms, and the tech stack used—including React Native, Express, PostgreSQL, and pgvector. It explains how specific features like the RAG-powered chatbot and AI content generation pipeline were engineered to support the platform's objectives.
- **Chapter 5** serves as the user manual, offering operational guides for both the learner mobile application and the administrative web

dashboard. It also provides a comprehensive installation guide, outlining prerequisites and environment configuration to ensure developers can successfully deploy and run the system.

- **Chapter 6** concludes the document by summarizing the project's achievements and findings. It also outlines potential future work, including the introduction of new user roles, planned empirical validation studies, the integration of social and cooperative learning features, and further refinements to pedagogical depth.

Chapter Two:

Background

2.1 The Mobile Learning Landscape

Mobile learning has become a primary mode of self-directed study, yet engagement decay is well documented across self-paced platforms. Dee et al. [1] report that fewer than 6% of MOOC registrants earn a certificate, with sharp drop-off in the first week; Kizilcec et al. [4] find that fewer than 25% of mobile learners return the day after first use. This persistence problem frames the design of Questify and motivates the rest of the chapter.

2.2 Gamification in Education

Gamification — applying game elements such as points, badges, and leaderboards to non-game contexts — has been studied extensively in education. In the first comprehensive review of empirical work, Hamari et al. [3] found that gamification generally produces positive effects, but the magnitude depends on the implementation context. Crucially, gamification works best when elements are integrated with pedagogy rather than added as cosmetic rewards. A complementary line of work by Ryan, Rigby, and Przybylski [5] explains why gamification can succeed: drawing on Self-Determination Theory, they show that well-designed game experiences satisfy three basic psychological needs — autonomy, competence, and relatedness. Questify maps its gamification mechanics onto these needs: XP and boss-battle quests serve competence; quest selection and avatar customization serve autonomy; leaderboards and shared challenges serve relatedness. Complementing this, Pedreira et al. [8] propose a specific architectural framework to standardise gamification implementation in software engineering.

2.3 Habit Formation and Streak Mechanics

A subset of gamification — daily streaks, daily challenges, and streak-freeze items — relies on habit formation rather than purely intrinsic motivation. Wood and R nger[7] show that repetition in a stable context shifts behavior from deliberate to automatic, making it easier to start and harder to stop; disruption is cognitively costly. Questify treats the daily streak as a primary engagement mechanism with a low ability threshold (a single short quest per day), milestone rewards at 7, 30, and 100 days, and streak-freeze items to absorb interruptions — a design aligned with the principle that habit-forming behaviors should remain sustainable under stress.

2.4 AI-Augmented Personalized Learning

Generative AI has opened a new design space for personalized learning. Guettala et al. [2] review the shift from rule-based to generative personalization and report measurable benefits in engagement and learning outcomes. The principal forms are AI-assisted content generation, conversational AI tutoring, and adaptive sequencing. Guettala et al. [2] also identify the dominant risks: factual hallucination, curriculum misalignment, and learner over-reliance. The next section examines a specific technique that addresses the first two.

2.5 Retrieval-Augmented Generation in Education

Retrieval-augmented generation (RAG) conditions an LLM's response on documents retrieved at query time, grounding the output in a specific corpus and reducing hallucination. Schilling et al. [6] directly compared RAG with prompt engineering and fine-tuning for a course-specific AI tutor, and found

that RAG produced the most curriculum-aligned responses while avoiding the data and engineering overhead of fine-tuning. Questify adopts this approach: instructor-uploaded material is embedded in PostgreSQL via pgvector, and the "course-ask" interface retrieves from the course's own corpus at query time. RAG is preferred over fine-tuning to keep the authoring loop fast and the system maintainable as the curriculum evolves.

2.6 Existing Similar Systems

Existing platforms cover parts of Questify's scope but not all of it. **Duolingo** and **Brilliant** demonstrate strong mobile gamification with streaks and leaderboards, but are limited to centrally curated content. **Khan Academy** offers structured courses and is moving toward AI tutoring, but its catalog is not easily extensible by external instructors. **Memrise** and **Anki** provide effective spaced repetition with minimal gamification. No widely deployed platform combines all three of: (a) layered gamification with habit-forming streaks, (b) instructor-led content authoring, and (c) AI tutoring grounded in course material — the gap Questify fills.

Chapter Three:

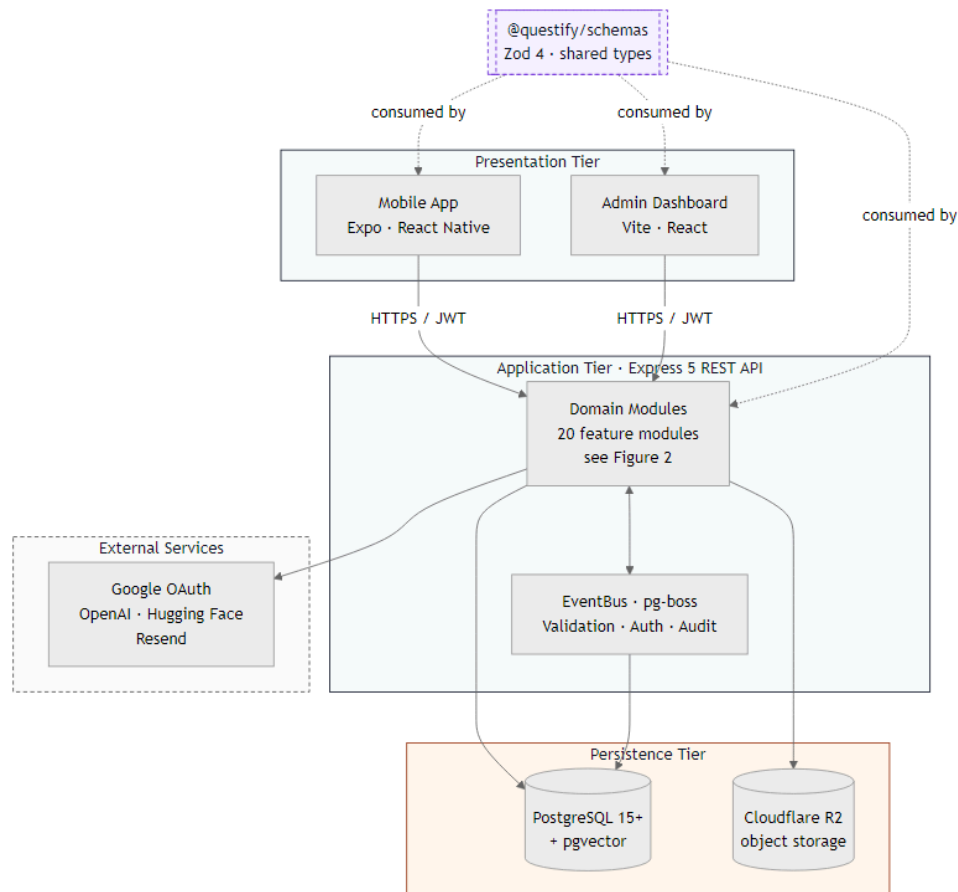
System Analysis &

Design

3.1 System Overview

3.1.1 System Architecture

The platform follows a classic three-tier client-server architecture with an event-driven core and a background job queue, supplemented by external services for AI, Google authentication, email, and object storage.



3.1.2 Functional Requirements

The functional requirements describe the main functions the system must provide. They are organised by capability and correspond to the modules implemented in the API, the mobile app, and the admin dashboard.

FR-1. Authentication and Account Management

ID	Requirement
FR-1.1	The system shall allow new users to register with email, username, password, first name, last name, and date of birth.
FR-1.2	The system shall send a one-time verification code to the user's email before completing registration (verify-before-register flow) using a transactional email service.
FR-1.3	The system shall allow registered users to log in with email/username and password and shall issue a JWT bearer token.
FR-1.4	The system shall allow users to sign in using Google OAuth 2.0 as an alternative to password-based login.
FR-1.5	The system shall securely store authentication tokens on the mobile device using platform-secure storage.
FR-1.6	The system shall support three user roles — STUDENT, LECTURER, and ADMIN — with role-based access control enforced at the API middleware layer.
FR-1.7	The system shall allow users to view and edit their own profile (bio, avatar, equipped items, featured badges) and account settings.

FR-1.8	The system shall allow users to log out and shall invalidate the active session.
---------------	--

FR-2. Course Content Authoring and Delivery

ID	Requirement
FR-2.1	The system shall organise learning content as a four-level hierarchy: Course → Section → Quest → Stage.
FR-2.2	The system shall support three stage types: INFORMATIONAL (reading material), MCQ (multiple-choice question), and BOSS BATTLE (multi-question challenge that gates progression).
FR-2.3	The system shall allow administrators to create, update, delete, reorder, publish, and unpublish courses, sections, quests, and stages.
FR-2.4	The system shall allow learners to browse published courses, view section and quest lists, and open individual stages.
FR-2.5	The system shall enforce a four-state progression lifecycle for content (LOCKED → UNLOCKED → IN_PROGRESS → COMPLETED) and a three-state lifecycle for courses (ENROLLED → IN_PROGRESS → COMPLETED).
FR-2.6	The system shall prevent a learner from completing a section or quest without first completing the previous one (linear gating).
FR-2.7	The system shall record per-stage mastery statistics (attempts, right count, wrong count, consecutive-right streak, mastered-at

	timestamp) and shall mark a stage as mastered once a configurable threshold is met.
--	---

FR-3. Enrollment

ID	Requirement
FR-3.1	The system shall allow a learner to enroll in a published course; the enrollment record is unique per (user, course) pair.
FR-3.2	The system shall track per-user course, section, and quest progress and shall report progress to the client on demand.
FR-3.3	The system shall allow administrators to view the roster of a course.

FR-4. Gamification — XP, Levels, Coins, and Streaks

ID	Requirement
FR-4.1	The system shall award XP and coins when a learner completes a quest or defeats a boss battle, based on the reward defined on the quest.
FR-4.2	The system shall recompute the learner's level from total XP using a level curve and shall emit a “user leveled up” event when the level changes.
FR-4.3	The system shall maintain an XP event log that records the source, amount, and optional course of every XP gain for analytics and leaderboards.
FR-4.4	The system shall track a daily login streak and shall reset the streak when a day is missed.
FR-4.5	The system shall emit a “streak at risk” event when a learner's streak is about to expire.

ID	Requirement
FR-4.6	The system shall support streak freeze items purchased in the shop: a freeze protects the next missed day from breaking the streak.

FR-5. Gamification — Badges, Achievements, and Challenges

ID	Requirement
FR-5.1	The system shall award badges based on metric-based thresholds across multiple tiers (BRONZE, SILVER, GOLD, LEGENDARY) within a family.
FR-5.2	The system shall allow a learner to feature up to a configurable number of earned badges on their profile, ordered by position.
FR-5.3	The system shall award achievements based on a rule definition expressed in JSON, with both general and course-scoped rules.
FR-5.4	The system shall publish challenges (daily or weekly) with a metric, threshold, optional filter (e.g. course), and XP/coin reward.
FR-5.5	The system shall track per-user challenge progress within a period, notify the user on completion, and grant rewards when the user claims the challenge.
FR-5.6	The system shall evaluate badges, achievements, and challenges in response to domain events through an event-bus-driven evaluator.

FR-6. Leaderboards

ID	Requirement
FR-6.1	The system shall compute and display three leaderboard kinds: Weekly Global , Monthly Global , and Weekly Per-Course .
FR-6.2	The system shall generate leaderboard snapshots periodically (background job) and shall allow learners to view their rank, score, and the surrounding entries.
FR-6.3	The system shall maintain weekly and monthly XP baselines so that period scores are computed from gain within the period.

FR-7. Shop, Inventory, and Avatar

ID	Requirement
FR-7.1	The system shall provide a virtual shop where learners can purchase items (avatar parts) with coins.
FR-7.2	The system shall maintain a per-user inventory of purchased items.
FR-7.3	The system shall allow a learner to equip at most one item per slot and to unequip a slot.

ID	Requirement
FR-7.4	The system shall allow administrators to create, update, delete, and feature shop items, with rarity and pricing metadata.

FR-8. Notifications

ID	Requirement
FR-8.1	The system shall deliver in-app notifications for relevant domain events (achievement earned, badge earned, etc..).
FR-8.2	The system shall allow a learner to list their notifications, retrieve the unread count, and mark notifications as read.
FR-8.3	The system shall allow administrators to compose and broadcast a notification to all users.

FR-9. Question Reports

ID	Requirement
FR-9.1	The system shall allow a learner to report a stage with a reason and an optional comment.
FR-9.2	The system shall expose the learner's own reports with their current status.

ID	Requirement
FR-9.3	The system shall provide an admin review workflow.
FR-9.4	The system shall automatically disable a stage when the number of open reports on it crosses a configured threshold and shall record the reason and timestamp.

FR-10. AI-Assisted Content Generation and RAG

ID	Requirement
FR-10.1	The system shall allow an administrator to upload a source document and create an AI content generation job for a chosen course.
FR-10.2	The system shall chunk the source document, generate embeddings using a local ML model, and store the chunks.
FR-10.3	The system shall call an LLM provider to generate a structured course outline (sections, quests, stages) and shall save the generated items as DraftStage records.
FR-10.4	The system shall allow the administrator to review each generated draft, edit it, approve it, or reject it; on approval, the draft shall be materialised as a real QuestStage in the course.
FR-10.5	The system shall expose a course chatbot (RAG) endpoint that takes a learner question, retrieves the top-k most similar chunks

ID	Requirement
	for the learner's enrolled course, and generates a grounded answer using the LLM.
FR-10.6	The system shall track per-chunk generation status, isolate failures per chunk, and allow partial completion of a job.

FR-11. Administration, Analytics, and Audit

ID	Requirement
FR-11.1	The system shall provide an admin dashboard (web) that allows operators to manage users, courses, sections, quests, stages, enrollments, shop items, achievements, badges, and challenges.
FR-11.2	The system shall compute and display platform analytics to administrators only.
FR-11.3	The system shall record an audit log entry for every privileged action.
FR-11.4	The system shall allow administrators to filter, search, and inspect the audit log.

FR-12. Cross-Cutting

ID	Requirement
FR-12.1	The system shall validate all incoming request bodies and query parameters against shared Zod schemas (single source of truth).
FR-12.2	The system shall publish domain events through an in-process typed event bus; modules shall subscribe to events rather than call each other directly.
FR-12.3	The system shall execute long-running or scheduled tasks (leaderboard snapshots, challenge crons, AI content generation) on a background job queue, not in the request path.
FR-12.4	The system shall return uniform, machine-readable error responses for client and validation errors.
FR-12.5	The system shall support image upload with automatic optimisation and storage in an S3-compatible object store.

3.1.3 Nonfunctional Requirements

Nonfunctional requirements define the quality attributes that the Questify system must satisfy and how each attribute is addressed by the architecture:

- **Performance:** API requests on the learner's hot path (course listing, stage playback, profile views) must return within 300 ms at the 95th percentile to keep the mobile experience feeling responsive. Progress is pre-computed rather than aggregated on every request, and heavy

computations such as leaderboard ranking are generated offline by background jobs rather than computed at read time.

- **Scalability:** The system must serve tens of thousands of concurrent learners without degrading response times. All computationally expensive work — leaderboard snapshots, challenge evaluation, AI content generation — runs on background workers outside the request-response cycle. The API process itself is stateless, so it can be scaled horizontally behind a load balancer as demand grows.
- **Security:** Every protected endpoint authenticates the caller through token-based verification. Role-based access control restricts sensitive operations to authorized users only, with three tiers (learner, content creator, administrator). Passwords are hashed using a modern adaptive algorithm, and all user input is validated before reaching the service layer to prevent injection and malformed-data attacks.
- **Reliability:** The API must remain available even when a background job fails. The worker system classifies failures into retryable and permanent categories, and the API serves traffic independently of worker health. Critical writes (progress, experience points, streak data) are committed atomically so partial failure cannot corrupt state. The mobile client caches data locally and retries automatically on network interruptions.
- **Maintainability:** The codebase must remain comprehensible as new features are added. Shared type definitions and validation rules are maintained in a single package consumed by all applications — the frontend, the backend, and the admin dashboard — ensuring consistency and reducing duplication. Each feature module follows a uniform structure, and new modules integrate through well-defined extension points rather than modifying existing code.
- **Testability:** Every application in the system must be testable at the unit and integration level. Pure logic — experience-point calculations,

progression rules, validation — lives in isolated modules that can be tested without the full runtime. Test data factories produce deterministic records for repeatable test runs.

- **Portability:** The mobile application runs on both iOS and Android from a single codebase using a cross-platform framework. The backend runs on any modern server environment, and the admin dashboard runs in any evergreen web browser.
- **Usability:** The learner-facing app must be intuitive enough to use with zero training. Gamification provides implicit guidance: content unlocks sequentially, rewards trigger celebratory animations and haptic feedback, and progress bars give clear next-action cues. The admin dashboard uses standard form patterns with inline validation, making it operable by non-technical content creators.
- **Data Integrity:** The system must never allow duplicate, orphaned, or inconsistent records. Constraints at the database level prevent double-enrollment in courses, overlapping challenge periods, and conflicting item configurations. All schema changes are tracked through version-controlled migrations.
- **Observability:** Every privileged action must be recorded for forensic review and compliance. The system logs each sensitive operation with the actor, action type, timestamp, outcome, and request context. An administrative interface provides a filterable audit-log viewer, and aggregate usage metrics are surfaced to help operators understand platform health.

3.1.4 System Users

A. Intended Users:

Questify serves three distinct user groups:

1. Learners (Students)

The primary audience. Learners interact exclusively through the mobile app. Their workflow is:

- Register an account, enroll in courses
- Progress through structured content: sections → quests → stages (informational reading, MCQ, boss battles)
- Earn XP, coins, level up, maintain daily streaks
- Earn badges, achievements, complete daily/weekly challenges
- Spend coins in the virtual shop on avatar items and streak freezes
- Appear on leaderboards, report problematic questions, ask the AI chatbot about course material

The entire learner experience is gamified — progression, rewards, and competition are the primary engagement drivers.

2. Content Creators (Lecturers)

Lecturers author and manage course material. They use the admin dashboard to:

- Create and publish courses, sections, quests, and stages
- Configure stage content (MCQ options, boss-battle parameters, informational text)
- Manage enrollments and review learner rosters
- Use the AI content generation pipeline: upload source documents, kick off AI job, review and approve generated draft stages
- Monitor basic course-level analytics

Lecturers do not have access to system-wide administration (users, roles, gamification rules).

3. System Administrators (Admins)

Full system operators with all Lecturer capabilities plus:

- Manage all user accounts (create, suspend, change roles)
- Define and manage gamification rules: badge definitions, achievement rules, challenge definitions
- Manage the virtual shop (item catalog, pricing, rarity)
- Review and resolve question reports
- Broadcast system-wide notifications
- View system analytics and the full audit log
- Administrators are the only users who interact with the audit, and system-wide analytics features.

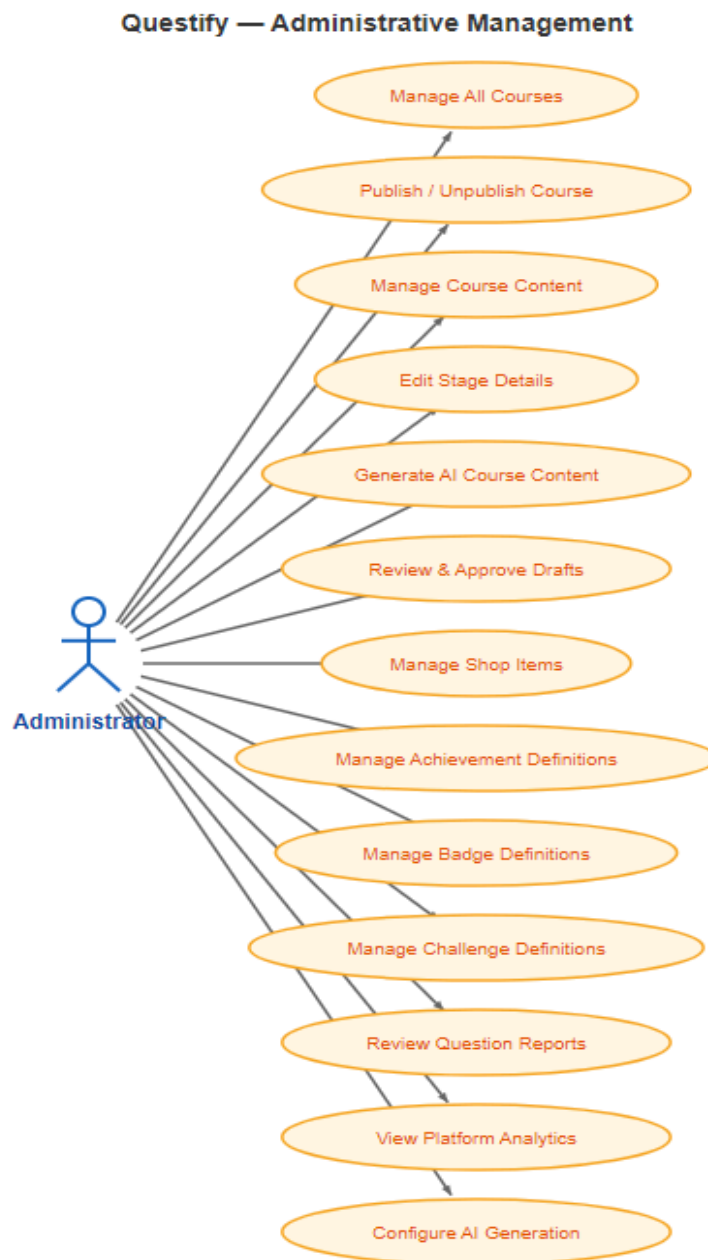
B. User Characteristics

User Group	Technical Skill Required	Domain Knowledge	Training needs
Learner	Minimal. Must be able to operate a smartphone and navigate a standard mobile app (tap, swipe, type). No prior exposure to gamified learning platforms is assumed.	None. The system is designed for general education; no specific prerequisite domain knowledge is required.	None. The app's progression (locked → unlocked → in progress → completed) is self-guiding. Rewards and visual feedback (animations, haptics) provide implicit guidance.
Lecturer	Low to moderate.	Must be familiar	Brief orientation

	Comfortable with standard web forms (point-and-click, drag-to-reorder, rich text input). Does not need to write code, handle APIs, or use a terminal.	with the subject domain they are creating content for.	session (~30 min) covering the course hierarchy (Course → Section → Quest → Stage), the three stage types, and the publish/unpublish workflow.
Adminstrator	Moderate. Comfortable with web dashboards, forms, data tables with filtering. For the AI content pipeline: must be able to upload a PDF and understand that generation runs asynchronously (background job).	Understands the platform's gamification model (badges, achievements, challenges, streaks, leaderboards) well enough to configure rules.	Orientation session (~1 hr) covering gamification rule configuration, AI content generation workflow (upload → review drafts → approve/reject), audit log inspection, and notification broadcasting.

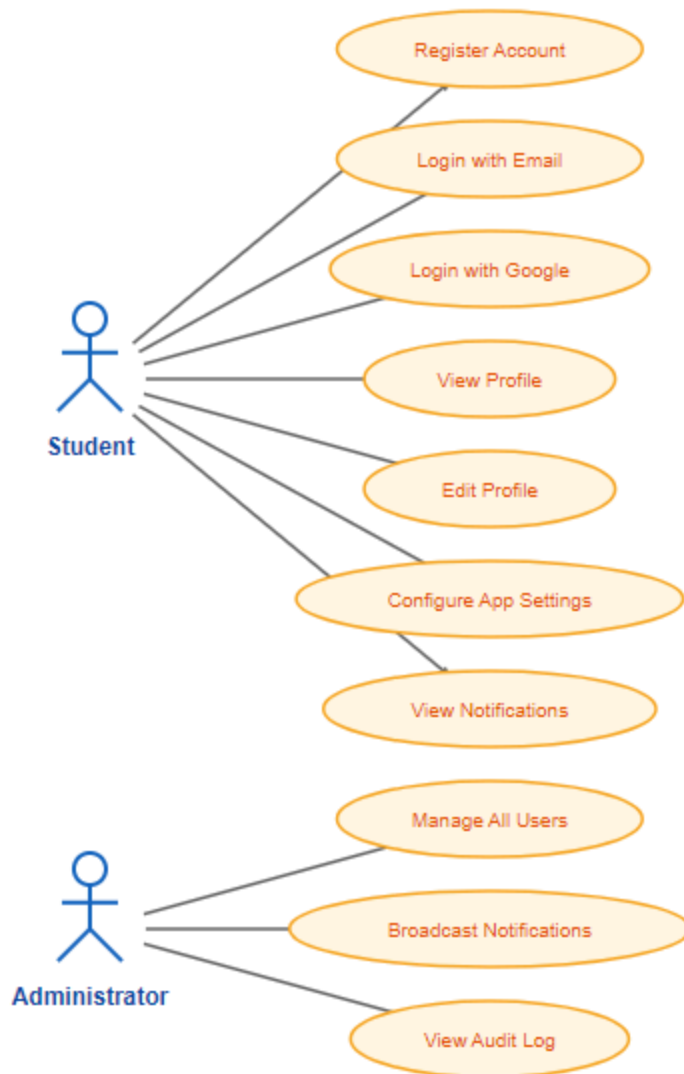
3.2 System Analysis & Design

3.2.1 Use Case Diagram

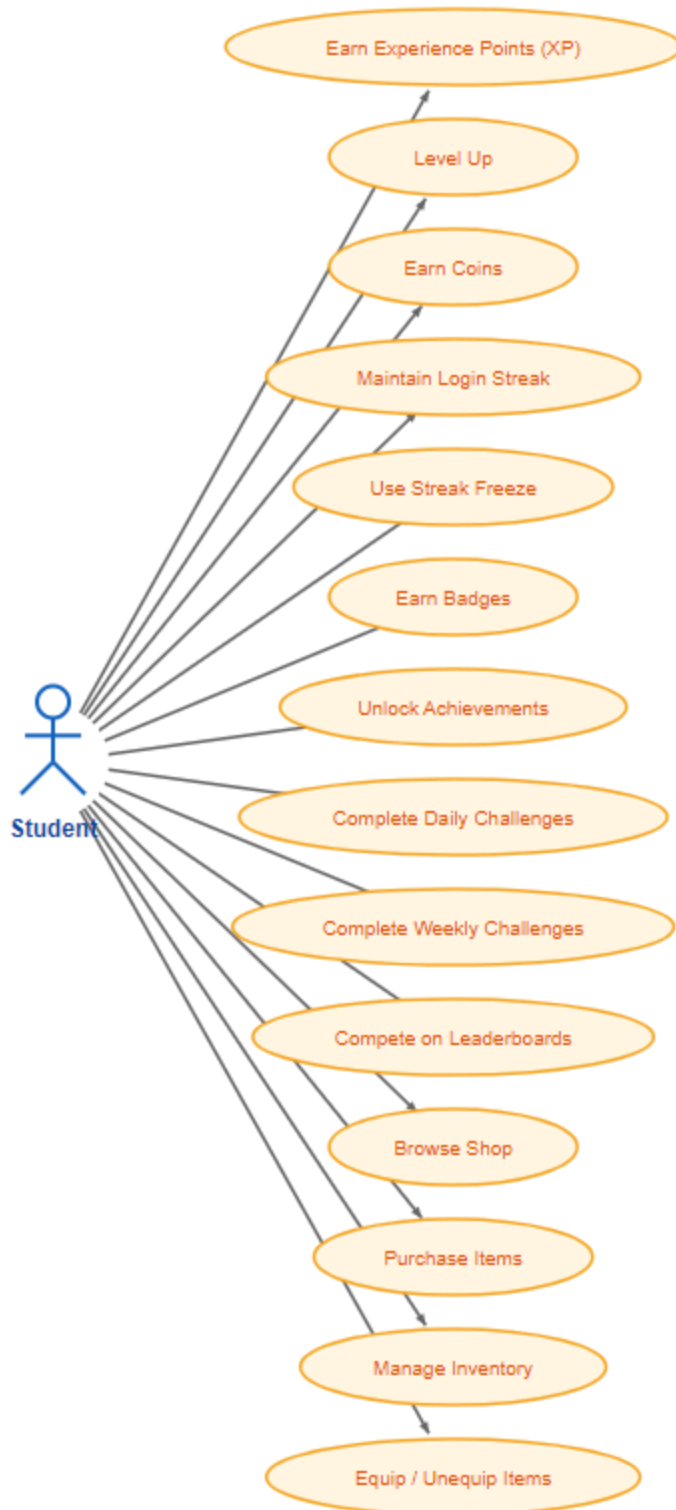


[Figure 3.1: Use Case Diagram]

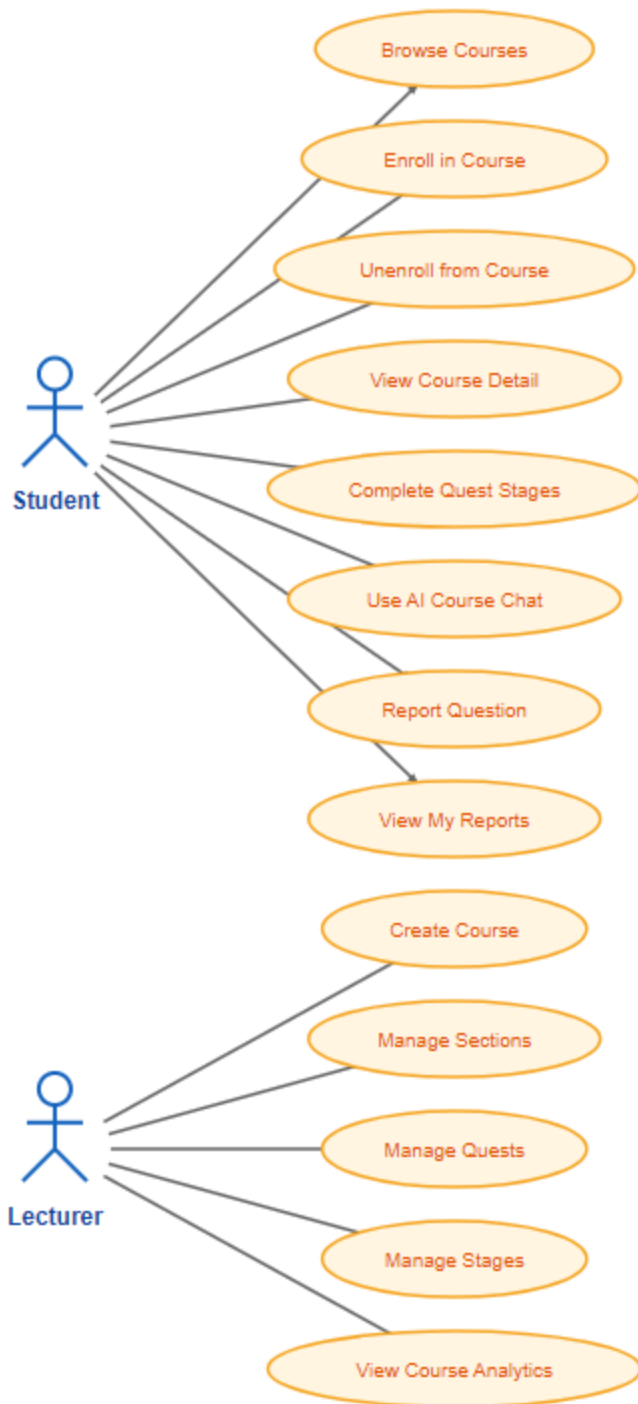
Questify — Authentication & Account



Questify — Gamification & Rewards

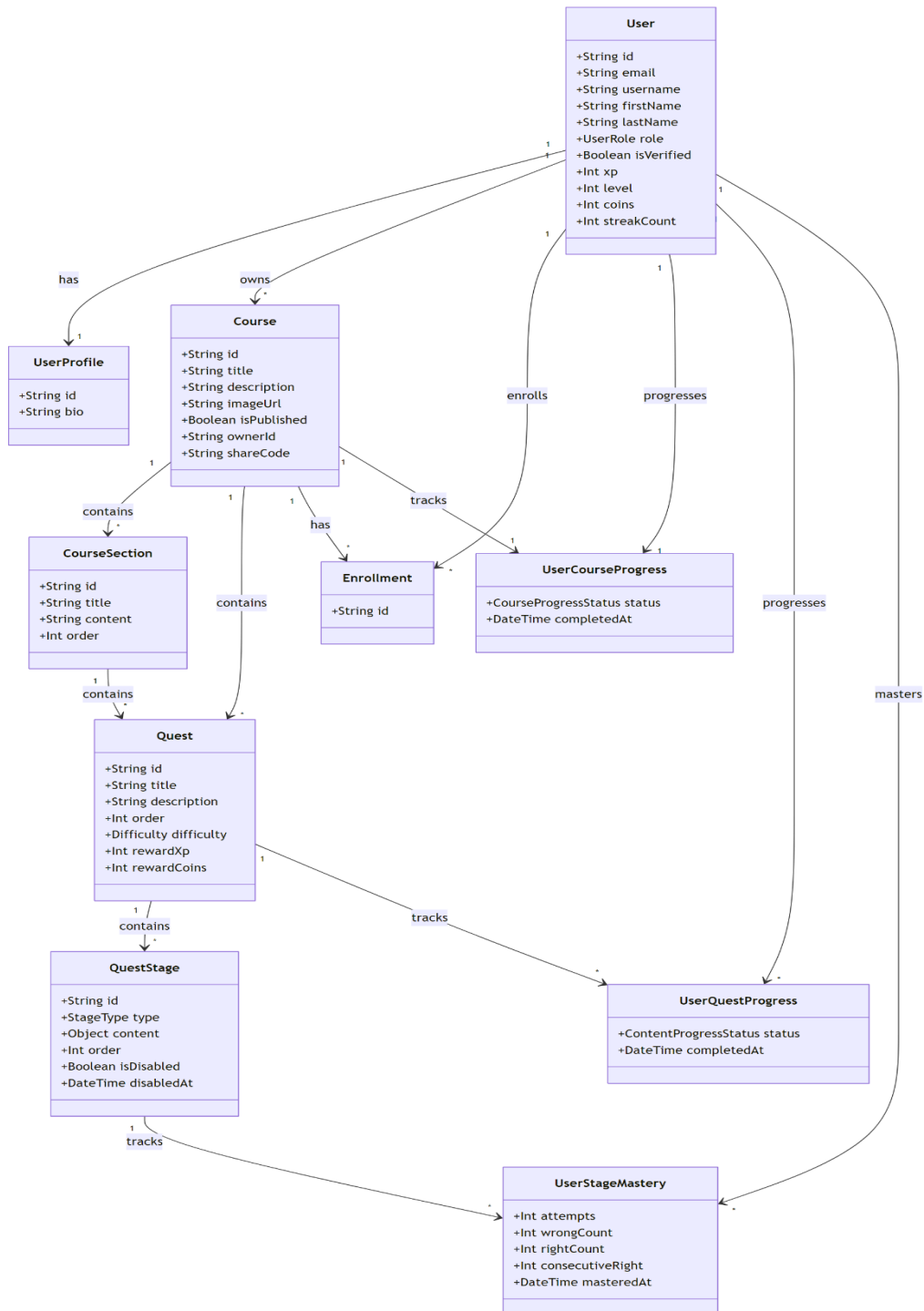


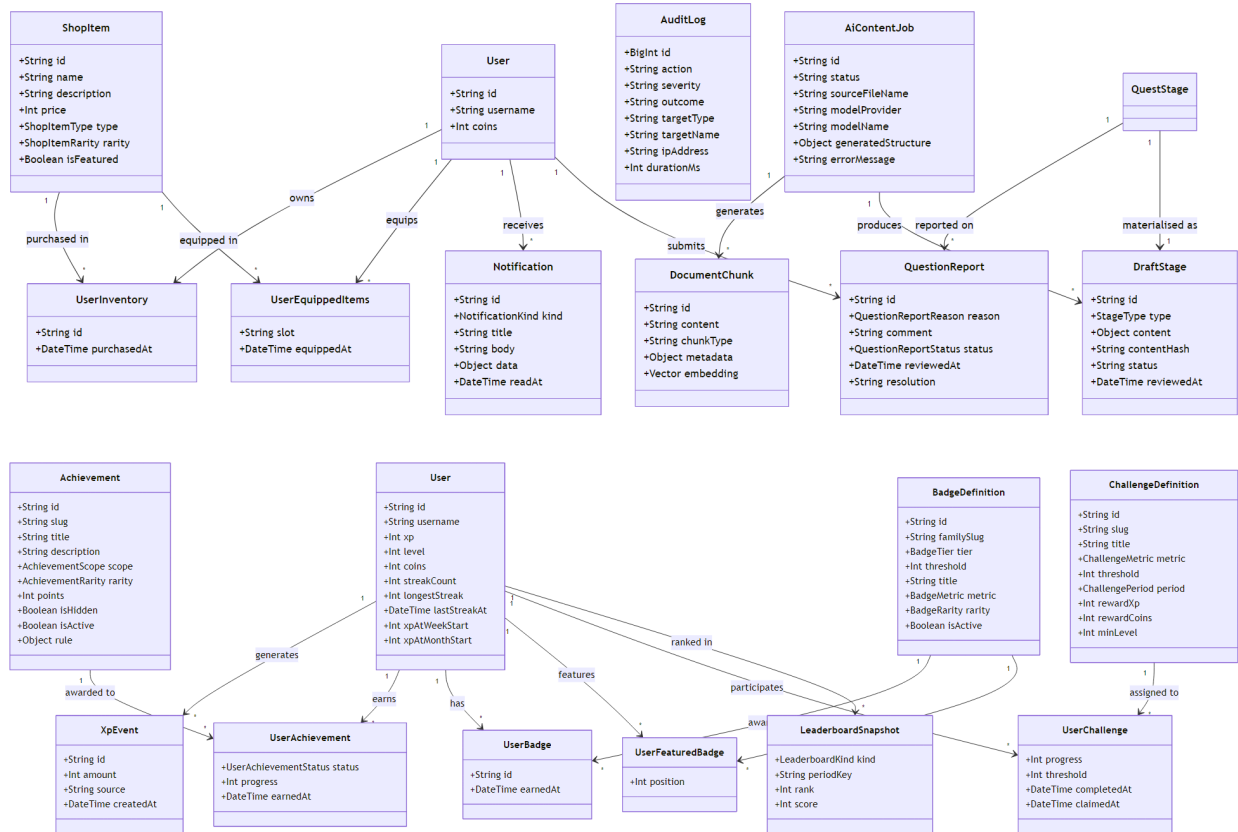
Questify — Student Learning Journey



3.2.2 Class Diagram

[Figure 3.2: Class Diagram]





Core Entities

- **User:** The central identity entity representing all actors (students, lecturers, admins) with authentication, personal info, and progression stats.
- **UserProfile:** A lightweight extension of User holding optional metadata like bios to keep the main user entity optimized.
- **Course:** The root container for learning content, grouping sections, quests, and stages with status management.
- **CourseSection:** A named subdivision of a course, such as a chapter or module, containing an ordered sequence of quests.

- **Quest:** A thematic grouping of stages acting as a lesson, defining rewards and difficulty.
- **QuestStage:** The atomic unit of learning (informational, MCQ, or boss battle) that triggers mastery tracking and quality reports.

Gamification & Economy

- **Achievement:** A rule-based reward entity defining conditions (e.g., counters) and rewards for general or course-scoped accomplishments.
- **BadgeDefinition:** A tiered classification for achievements (e.g., Login Streak) with rarity and point weightings.
- **ChallengeDefinition:** Time-boxed periodic goals (daily/weekly) with specific metrics and rewards to drive engagement.
- **ShopItem:** A purchasable virtual good (cosmetic avatar parts or consumables like streak freezes).
- **LeaderboardSnapshot:** Pre-computed views of student rankings generated by background workers for efficient querying.

Observability & Content Pipeline

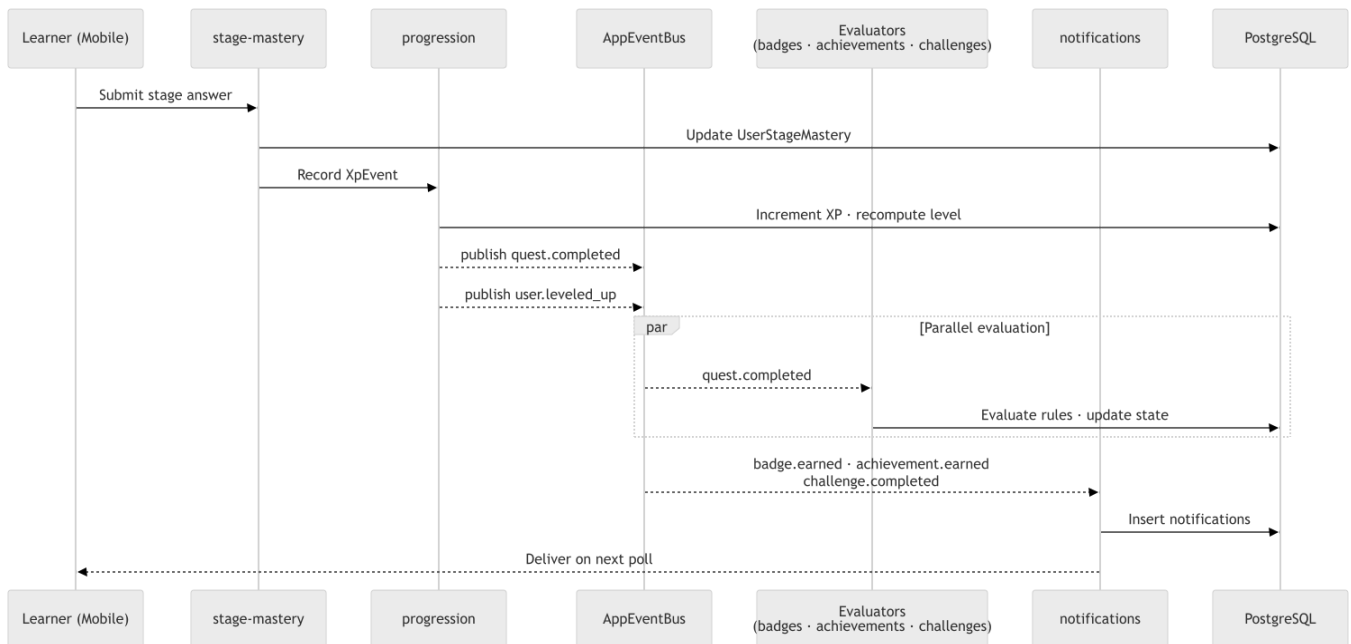
- **AuditLog:** An immutable record of all privileged actions, capturing the actor, action type, and performance context.
- **AiContentJob:** An orchestration record managing the end-to-end AI generation pipeline from document ingestion to stage creation.

- **DocumentChunk:** Text fragments of source documents stored with vector embeddings to enable retrieval-augmented generation (RAG).
- **DraftStage:** A generated learning stage awaiting administrative review before being materialized into a published QuestStage.

3.2.3 Sequence Diagrams

The completion of a quest fans out into a parallel set of evaluators through the event bus. This is the central pattern of the gamification subsystem and is what allows badges, achievements, challenges, notifications, and the audit log to evolve independently of the modules that emit events.

[Figure 3.3: Sequence Diagram]



3.2.4 Database Diagram

[Figure 3.4: Database Diagram]

Table	Description	Key Fields
Identity and Authentication		
users	Central user entity for all platform actors (students, lecturers, admins). Stores credentials, OAuth identifiers, and gamification attributes (XP, level, coins, streaks).	id, email, username, role, xp, level, coins, streakCount
verification_tokens	Email verification codes with expiry tracking.	id, email, codeHash, expiresAt, purpose
user_profiles	One-to-one extension of user profiles for biographical data.	id, userId (FK), bio
Course Content Hierarchy		
courses	Top-level educational content container owned by a creator.	id, title, description, imageUrl, isPublished, ownerId (FK), shareCode
course_sections	Ordered subdivisions within a course.	id, courseId (FK), title, content, order
quests	Learning units within a section with difficulty and reward settings.	id, sectionId (FK), courseId (FK), title, order, difficulty, rewardXp, rewardCoins
quest_stages	Atomic interactive stage within a quest. Types: informational text, MCQ, or boss battle. Content stored as JSON.	id, questId (FK), courseId (FK), type, content (JSON), order
Enrollment and Learner Progress		

Table	Description	Key Fields
enrollments	Records user-course enrollment relationships.	id, userId (FK), courseId (FK)
user_course_progress	Tracks course-level completion status per user.	id, userId (FK), courseId (FK), status, completedAt
user_section_progress	Tracks section-level progress per user.	id, userId (FK), sectionId (FK), status, completedAt
user_quest_progress	Tracks quest-level progress per user.	id, userId (FK), questId (FK), status, completedAt
user_stage_mastery	Detailed per-stage performance metrics including attempts, correct/incorrect counts, and mastery timestamps.	id, userId (FK), stageId (FK), courseId (FK), attempts, wrongCount, rightCount, masteredAt
Shop and Inventory		
shop_items	Virtual goods available for purchase — avatar cosmetics and functional items.	id, name, description, price, type, rarity, metadata (JSON)
user_inventory	Records purchased items per user.	id, userId (FK), itemId (FK), purchasedAt
user_equipped_items	Tracks currently equipped items by avatar slot; enforces one item per slot.	id, userId (FK), itemId (FK), slot, equippedAt
Gamification — Achievements, Badges, and Challenges		

Table	Description	Key Fields
achievements	Achievement templates with earn rules, rarity, scope (general or course-specific).	id, slug, title, description, rule (JSON), scope, rarity, points, isHidden
user_achievements	Per-user achievement progress and earnings.	id, userId (FK), achievementId (FK), status, progress, earnedAt
badge_definitions	Tiered badge families (bronze through legendary) keyed to specific metrics.	id, familySlug, tier, threshold, title, metric, rarity, points
user_badges	Records earned badges per user.	id, userId (FK), badgeDefinitionId (FK), earnedAt
user_featured_badges	Badges a user chooses to display on their profile.	id, userId (FK), badgeDefinitionId (FK), position
challenge_definitions	Time-bound daily or weekly challenges with configurable metrics and rewards.	id, slug, title, description, metric, threshold, period, rewardXp, rewardCoins
user_challenges	Per-user challenge attempts within a time period.	id, userId (FK), challengeDefinitionId (FK), periodStart, periodEnd, progress, threshold, completedAt
Notifications, Reports, and Audit		

Table	Description	Key Fields
notifications	User-facing notification messages categorized by event kind.	id, userId (FK), kind, title, body, data (JSON), readAt
question_reports	User-reported issues with stage content; lifecycle from open through review to resolution.	id, userId (FK), stageId (FK), reason, status, reviewedById (FK)
audit_logs	Comprehensive administrative audit trail for all significant platform actions.	id (BigInt), actorId (FK), action, severity, outcome, targetType, targetId, metadata (JSON)
Leaderboards and XP Events		
xp_events	Append-only log of all experience point transactions.	id, userId (FK), courseId, amount, source
leaderboard_snapshots	Periodic ranking snapshots (weekly global, monthly global, course-weekly) for efficient queries.	id, kind, periodKey, userId (FK), rank, score, courseId
RAG and AI Content Pipeline		
ai_content_jobs	Tracks AI-powered course content generation pipeline runs including file metadata and model selection.	id, courseId (FK), status, modelName, sourceFileName, generatedStructure (JSON)
document_chunks	Text chunks with 384-dimensional vector embeddings for semantic search via pgvector.	id, content, embedding (vector(384)), courseId (FK), chunkType, metadata (JSON)

Table	Description	Key Fields
draft_stages	AI-generated quest stages pending human review; links to final stage upon approval.	id, jobId (FK), type, content (JSON), contentHash, status, approvedAsStageId (FK)

Chapter Four:

System Implementation

4.1 Questify's Purpose

Questify is a gamified learning management system designed to enhance student engagement through game mechanics. The platform is implemented as a three-tier architecture comprising a React Native mobile application for students, a React-based administrative web panel for educators, and an Express REST API backend with a PostgreSQL database. This chapter details the implementation of each system function, the algorithms and techniques employed, and the key technologies leveraged during development.

4.2 System Functions

The system is organized into a set of functional modules, each responsible for a distinct domain within the platform. The following subsections describe the purpose and implementation of each module.

4.2.1 Course Content Management

This module manages the hierarchical structure of educational content, organized as courses, sections, quests, and stages. A course contains multiple sections, each section contains multiple quests, and each quest is composed of stages representing individual learning activities. Stage types include informational content, multiple-choice questions (MCQs), and boss battles. The module provides full CRUD operations for each content entity, along with reordering capabilities through a dedicated content ordering service.

The admin panel provides interfaces for creating and editing courses, managing sections and quests, defining stage content with various question types, and setting quest difficulty levels (Easy, Medium, Hard). Each quest stores reward values for experience points and coins, calculated dynamically based on its stage composition and difficulty multiplier.

4.2.2 User Progression

The progression module manages the linear unlocking of educational content for students. When a user enrolls in a course, the system initializes progress records for the first section and quest. Completion flows sequentially: completing a quest unlocks the next quest within the same section; completing all quests in a section marks the section as completed and unlocks the first quest of the next section; completing all sections marks the course as completed. The module enforces gating logic that prevents access to locked content, verifying that prerequisite quests and sections are completed before allowing progression. Progress is tracked at three granularity levels: course-level, section-level, and quest-level, each maintaining status indicators and timestamps.

4.2.3 Stage Mastery Tracking

Stage mastery tracking records detailed attempt data for each stage a student interacts with. The system tracks total attempts, right answers, wrong answers, consecutive correct answers, and the timestamp of the first wrong answer. When a student achieves the mastery threshold of two consecutive correct answers, the stage is marked as mastered. This data supports the identification of weak areas, allowing students to focus their revision on stages where they have demonstrated difficulty.

4.2.4 Challenges

The challenges module implements time-bound incentive mechanics. Challenges are defined by administrators with a target metric (such as quests completed, logins, boss battles defeated, courses completed, or total XP earned), a numerical threshold, and a period (daily or weekly). When a user performs an action that matches a challenge metric, the challenge evaluator processes the event, increments the user's progress toward the relevant challenge, and checks whether the threshold has been met. Upon completion, rewards are granted and a challenge-completed event is published. A self-healing mechanism automatically creates missing progress records when an event fires before the scheduled creation of user challenge records.

4.2.5 Achievements

The achievements module provides a rule-based system for awarding one-shot and counter-based achievements. One-shot achievements are granted the first time a specific event occurs, while counter achievements require a user to reach a numerical threshold over time. The evaluator employs atomic conditional updates to guarantee idempotency under concurrent event processing, ensuring that a single achievement is never awarded twice. For login-based counter achievements, a daily deduplication mechanism prevents multiple logins on the same calendar day from being counted multiple times.

4.2.6 Badges

The badge system provides tiered progression within defined badge families. Each badge family (such as login streak, quests completed, courses completed, bosses defeated, or total XP) has four tiers: Bronze, Silver, Gold, and Legendary, with increasing thresholds. When a relevant event occurs, the badge evaluator computes the user's current metric value, determines the highest qualifying tier, and awards any unearned badges in the progression path. Each badge carries a rarity level and point value that contributes to the user's total experience score.

4.2.7 Leaderboards

The leaderboard module provides three ranking dimensions: weekly global, monthly global, and course-specific weekly rankings. Rankings are computed based on experience points earned within the period. Global leaderboards use snapshot-style scores derived from the difference between the user's total XP and their XP recorded at the start of the period. Course leaderboards aggregate XP events specific to that course. Leaderboard data is served via snapshot tables that are periodically computed and cached.

4.2.8 Shop and Inventory

The shop module enables students to purchase cosmetic avatar items and consumable streak freeze items using in-app currency. Avatar items are categorized into three equippable slots: head, body, and lower body. The purchase flow validates coin balance, enforces unique ownership for non-consumable items, and enforces a maximum cap on streak freeze inventory. Upon purchasing an equippable item, the system automatically equips it if the corresponding slot is empty. The inventory module tracks all owned items, and the equipment system manages currently equipped items.

4.2.9 Streak Freeze

The streak freeze module implements a safety net mechanic that protects a user's login streak when they miss a day. Each missed calendar day consumes one streak freeze item from the user's inventory. The system employs a per-day consumption model: if the user has sufficient freezes to cover all missed days since their last activity, the streak is preserved and the freezes are consumed; if there are insufficient freezes, the streak resets. Streak status is evaluated both on application startup and via a nightly cron job.

4.2.10 RAG-Powered Chatbot

The retrieval-augmented generation (RAG) module provides a course-specific question-answering chatbot. PDF lecture materials are ingested by extracting text and splitting it into chunks of approximately one thousand characters with overlap. Each chunk is embedded locally using a sentence transformer model and stored with a vector representation. When a student asks a question, the system embeds the query and performs a vector similarity search, retrieving the ten most relevant passages as context for a large language model to generate a concise answer.

4.2.11 AI Content Generation

The AI content generation module automates the creation of quest stages from uploaded source materials. An administrator uploads a PDF document, which is parsed and split into chunks. Each chunk is processed individually by a large

language model to produce a structured set of stages, including informational slides and multiple-choice questions. Generated drafts enter a reviewable state where administrators can review, edit, approve, or reject each draft. A bulk approval mode with auto-resolution automatically creates or resolves the appropriate sections and quests based on the document structure. Recent methodologies for automated assessment have advanced rapidly; Biancini et al. [9] and Mucciaccia et al. [10] have both demonstrated effective frameworks for generating multiple-choice questions from educational content using LLMs, which supports the viability of our own AI content generation pipeline.

4.2.12 Notifications

The notification module provides an in-app notification system that alerts users to important events within the platform. Notifications are created for achievement earnings, badge awards, challenge completions and claims, streak events, course publications, and stage masteries. The notification dispatcher subscribes to the platform's event bus and creates notification records asynchronously, ensuring that notification failures never disrupt the primary event flow.

4.2.13 Authentication and Authorization

The authentication module implements JWT-based authentication with support for email-password login and Google OAuth integration. Passwords are hashed using bcrypt before storage. Authorization is enforced at the route level using role-based guards that restrict access to administrator-only endpoints. The mobile application supports persistent sessions via secure token storage, and the admin panel uses a separate login flow with session management.

4.2.14 Audit Logging

The audit log module records administrative actions for security and compliance purposes. Each audit entry captures the acting administrator, the action performed, the target entity type and identifier, a description of the

change, and a timestamp. Audit logs are queryable and paginated, enabling administrators to review the history of changes made within the platform.

4.2.15 Analytics and Reporting

The analytics module provides aggregate insights into platform usage and course performance. The administrative dashboard presents course-level analytics including enrollment counts, quest completion rates, and student engagement metrics. The question reporting module allows students to flag problematic questions, and administrators can review reports, disable flagged stages, and manage report resolution workflows.

4.3 Techniques and Algorithms

This section describes the key algorithms and implementation techniques employed in the system.

4.3.1 Event-Driven Architecture

The system implements a publish-subscribe event bus that enables loose coupling between modules. Core domain events such as quest completion, login, level-up, and challenge completion are published to the bus. Downstream modules including the challenge evaluator, achievement evaluator, badge evaluator, and notification dispatcher subscribe to relevant events and react independently. This decoupling allows new features to be added without modifying existing event producers.

4.3.2 Stage Mastery Algorithm

Stage mastery is determined using a consecutive-correct tracking algorithm. Each stage attempt records whether the answer was correct and updates a running count of consecutive correct answers. When the consecutive count reaches a threshold of two, the stage is marked as mastered. Any incorrect

answer resets the consecutive count to zero. Mastery status is persisted and once achieved, subsequent attempts are no-ops to prevent regression.

4.3.3 Leveling Algorithm

User level is computed using a square-root-based formula: level equals the floor of the square root of experience points divided by one hundred, plus one. This produces a gradually slowing leveling curve where early levels require few points and higher levels require progressively more. Rank titles (Novice, Pathfinder, Adventurer, Legend) are assigned based on level thresholds.

4.3.4 Retrieval-Augmented Generation Pipeline

The RAG pipeline implements a complete retrieval-augmented generation workflow for course-specific question answering. The ingestion phase uses recursive character text splitting to divide PDF content into overlapping chunks of approximately one thousand characters. Each chunk is embedded into a high-dimensional vector space using a locally-running sentence transformer model via ONNX runtime. At query time, the student's question is embedded and a vector similarity search is performed using cosine distance via PostgreSQL's pgvector extension. The top ten most similar chunks are retrieved and passed as context to a large language model with instructions to answer solely from the provided material.

4.3.5 Streak Freeze Algorithm

The streak freeze algorithm implements a per-day consumption model. When a user returns after inactivity, the system computes missed calendar days by comparing the last activity date to the current date in UTC. Each missed day requires one streak freeze item from inventory. If sufficient freezes exist, they are consumed and the streak is preserved. If insufficient, the streak resets. On the first day of inactivity with no freezes, the user retains the chance to engage before the streak breaks.

4.3.6 Idempotent Achievement Evaluation

Achievement evaluation uses a two-phase atomic update pattern. Progress is incremented using a conditional database update that only applies when the achievement status is not already earned. When progress reaches the threshold, a second conditional update attempts to transition the status to earned, guarded by the same status check. Only the first concurrent request to successfully update the status proceeds to grant rewards, preventing double-rewarding under high concurrency.

4.3.7 Leaderboard Snapshot Pattern

Leaderboards use a snapshot pattern rather than computing rankings on every read. A background job periodically computes current scores for all active users and stores them in a leaderboard snapshot table with precomputed ranks. Reads from the snapshot table are efficient single-table queries. Three leaderboard kinds are maintained: weekly global, monthly global, and course-specific weekly.

4.3.8 Content Ordering Service

Content ordering implements a position-management system for ordered lists. Each content item maintains an integer order field within its parent scope. Inserting or moving items shifts surrounding positions within a database transaction, ensuring that ordering invariants are maintained atomically and no two items share the same order value within a parent container.

4.3.9 Job Queue Processing

Long-running and asynchronous tasks are managed through a PostgreSQL-based job queue system. AI content generation, leaderboard computation, and streak freeze cron tasks are dispatched as jobs with configurable retry limits, backoff delays, and error handling. The queue provides visibility into pending, active, failed, and completed jobs for administrative monitoring.

4.4 Technologies Used

This section describes the key technologies leveraged in the implementation of Questify.

4.4.1 Expo and React Native

The mobile application is built with Expo SDK 56, providing a managed development environment for React Native. Expo Router handles file-based navigation, Expo Secure Store manages token persistence, and Expo Haptics provides tactile feedback. The application uses react-native-reanimated for animations and react-native-gesture-handler for touch interactions.

4.4.2 Express and Prisma

The API backend is built with Express 5, offering improved async error handling and middleware architecture. Data access is managed through Prisma ORM version 7, providing type-safe database queries, automated migrations, and a declarative data model definition with PostgreSQL.

4.4.3 PostgreSQL with pgvector

PostgreSQL serves as the primary database, extended with the pgvector extension for vector similarity search. This enables efficient cosine distance queries on embedding vectors directly within the database, eliminating the need for a separate vector database service.

4.4.4 pg-boss Job Queue

pg-boss is used as the job queue system, leveraging PostgreSQL as the job store without requiring a separate message broker. It provides reliable job scheduling, retry mechanisms, and concurrency control for background tasks.

4.4.5 AI SDK and Language Models

The Vercel AI SDK provides a unified interface for interacting with large language models. OpenRouter serves as the primary LLM gateway, offering access to multiple models through a single API. Structured generation with schema validation is used for content generation tasks.

4.4.6 Local Embeddings with Transformers.js

Text embeddings are generated locally using the Xenova Transformers library via ONNX runtime. The all-MiniLM-L6-v2 sentence transformer model produces 384-dimensional embeddings without requiring external API calls, reducing latency and operational costs.

4.4.7 LangChain Text Splitters

LangChain's RecursiveCharacterTextSplitter divides text on separator boundaries to create chunks that respect document structure. Chunks of approximately one thousand characters with overlap preserve context for both RAG retrieval and AI content generation.

4.4.8 TanStack Query and Zustand

TanStack Query manages client-side data fetching with automatic caching, background refetching, and request deduplication for both mobile and admin applications. Zustand handles lightweight client-side state management for UI preferences and temporary data.

4.4.9 Radix UI and Recharts

The admin panel uses Radix UI primitives for accessible dialog, dropdown, select, and tab components. Recharts provides SVG-based charting for the analytics dashboard, including line charts, bar charts, and pie charts for data visualization.

4.4.10 Zod Schema Validation

Zod provides schema definition and validation across all application tiers. A shared schemas package ensures consistent input validation between the API server and client applications, with TypeScript type inference from schema definitions.

4.4.11 Supporting Technologies

Additional technologies include bcryptjs for password hashing, JSON Web Tokens for stateless authentication, Passport.js for Google OAuth, Resend for transactional email, Sharp for image processing, and AWS SDK for file storage. The development toolchain includes TypeScript, Vitest for testing, and ESLint for code quality.

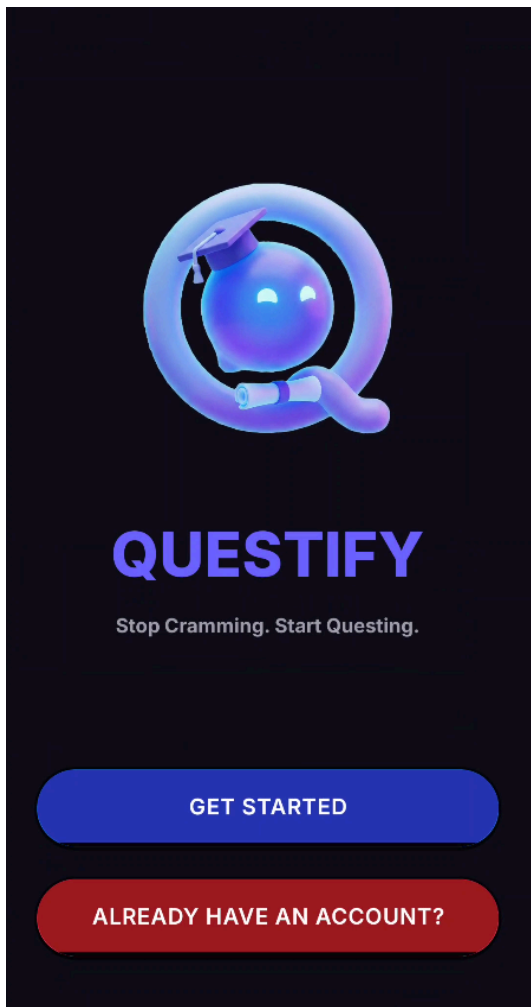
Chapter Five:

System Manual

5.1 Operation Manual

Mobile App:

1. Open the app, you will be greeted with the home page



2. Press “Get Started” to register a new account.

3. Fill the fields with your data and press “Send Verification Code”

Let us set up your account details before you start questing.

FIRST NAME

First Name

LAST NAME

Last Name

USERNAME

Username

EMAIL ADDRESS

you@example.com

PASSWORD

Minimum 6 characters

CONFIRM PASSWORD

Re-enter your password

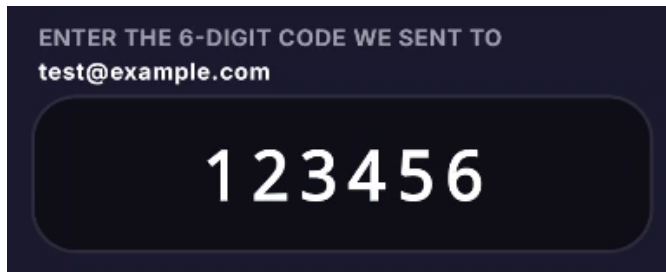
DATE OF BIRTH

2000-01-01

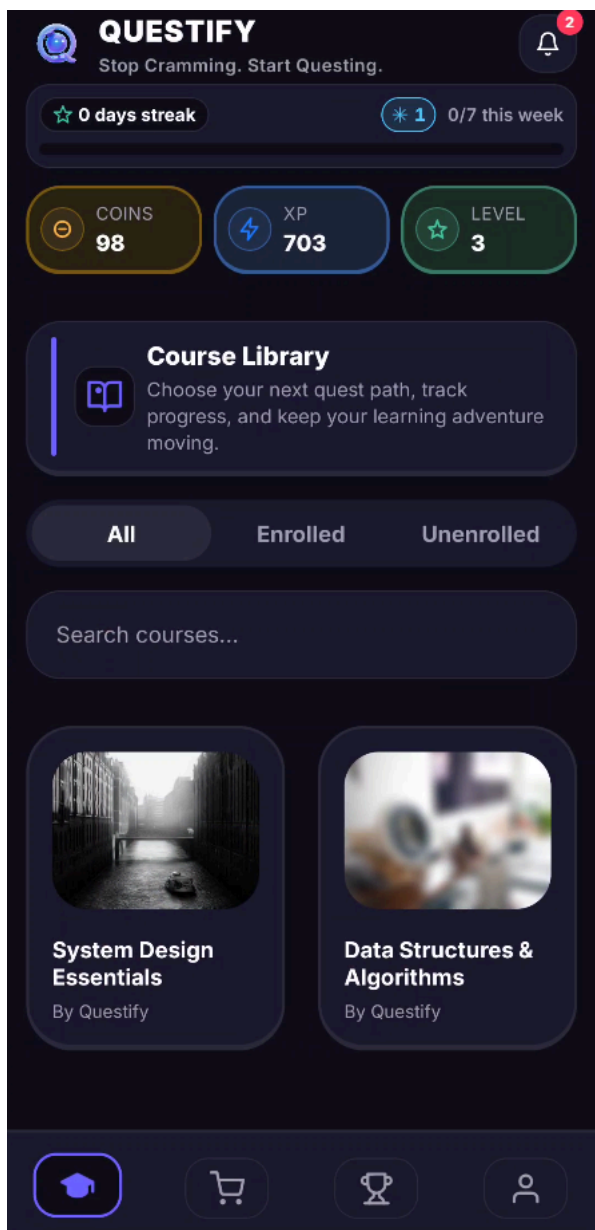
CHANGE

SEND VERIFICATION CODE

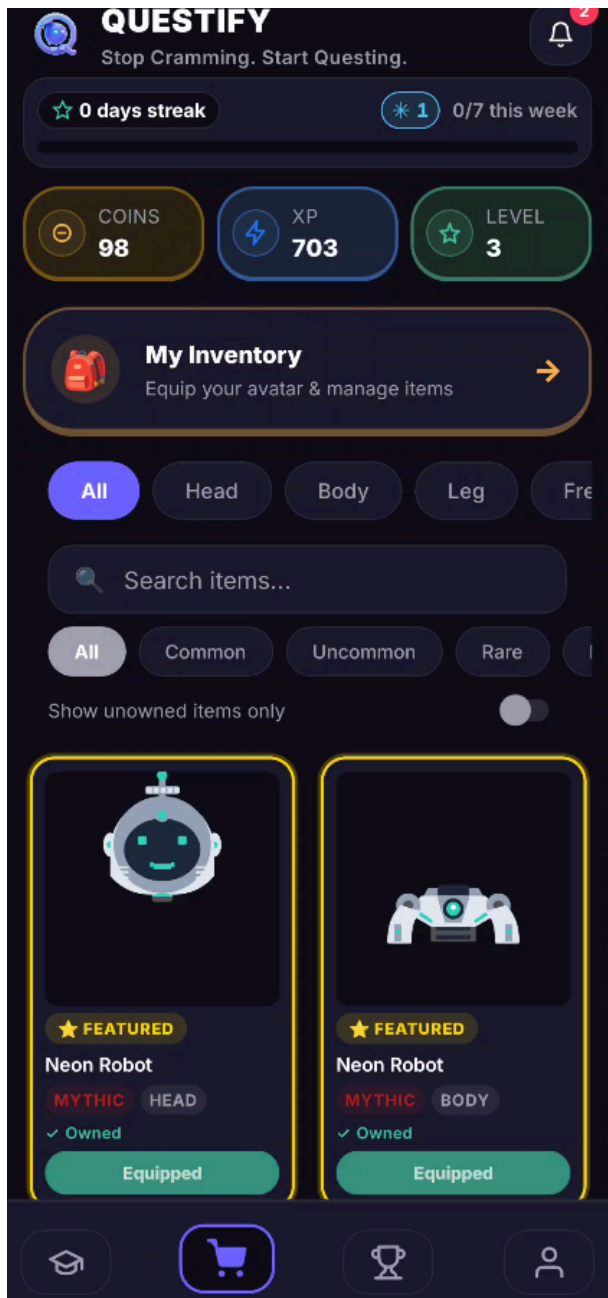
4. You will receive a one-time code sent to your email, type the code in the field that appears.



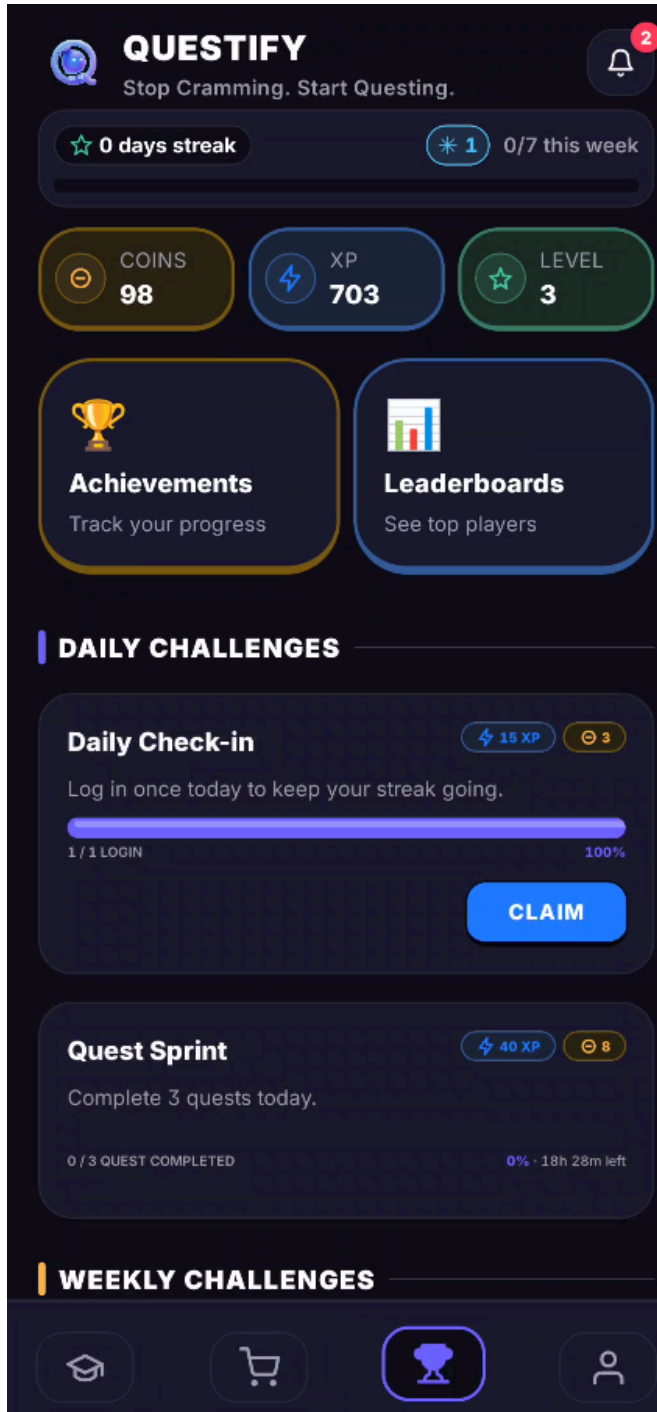
5. You will be redirected to logged-in users' homepage.



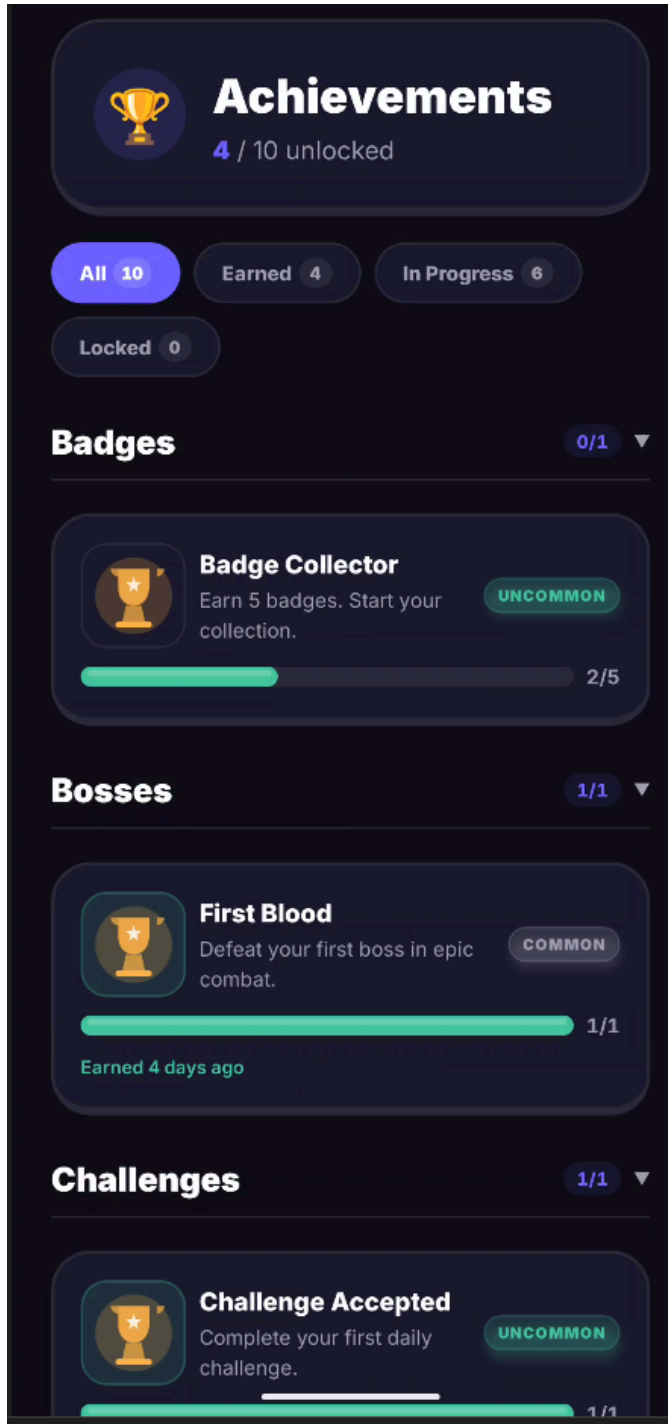
6. Press on your desired course and press “Enroll Now” to enroll.
7. Now you can progress through the course content.
8. Press on the shop icon at the bottom of the homepage to go to the shop screen where you can buy items for your avatar.



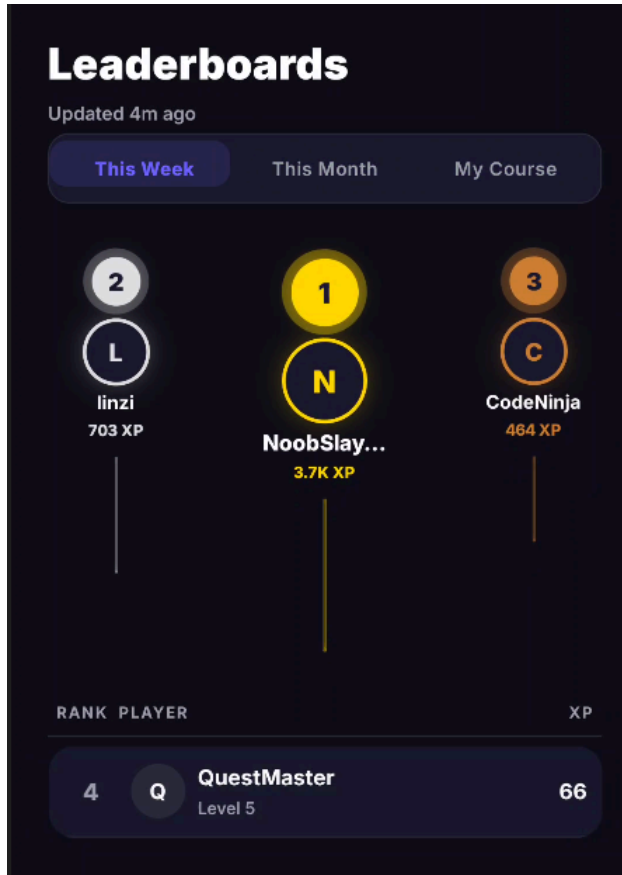
9. Press on the trophy icon at the bottom of the homepage to go to the challenges screen where you can complete daily and weekly challenges for rewards.



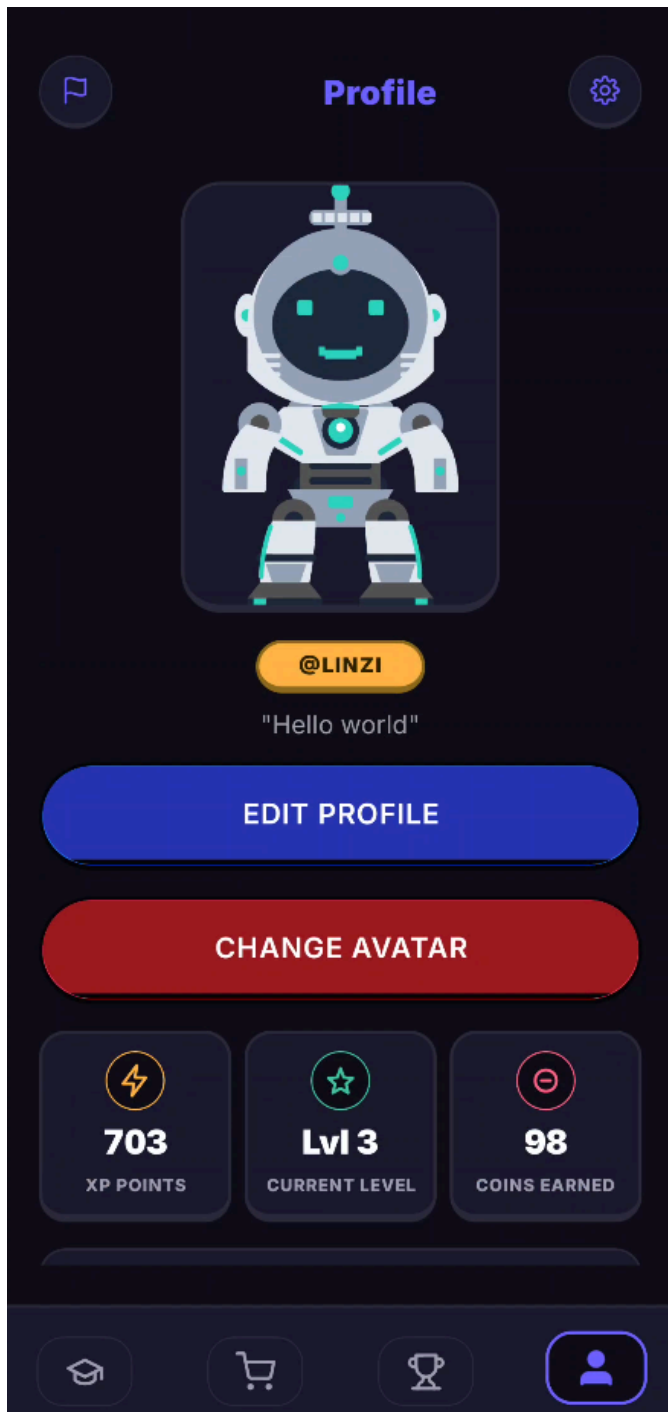
10. In the challenges screen, you will find “Achievements” button to see all the list of achievements. Some achievements won't be visible to you as they are locked behind specific courses.



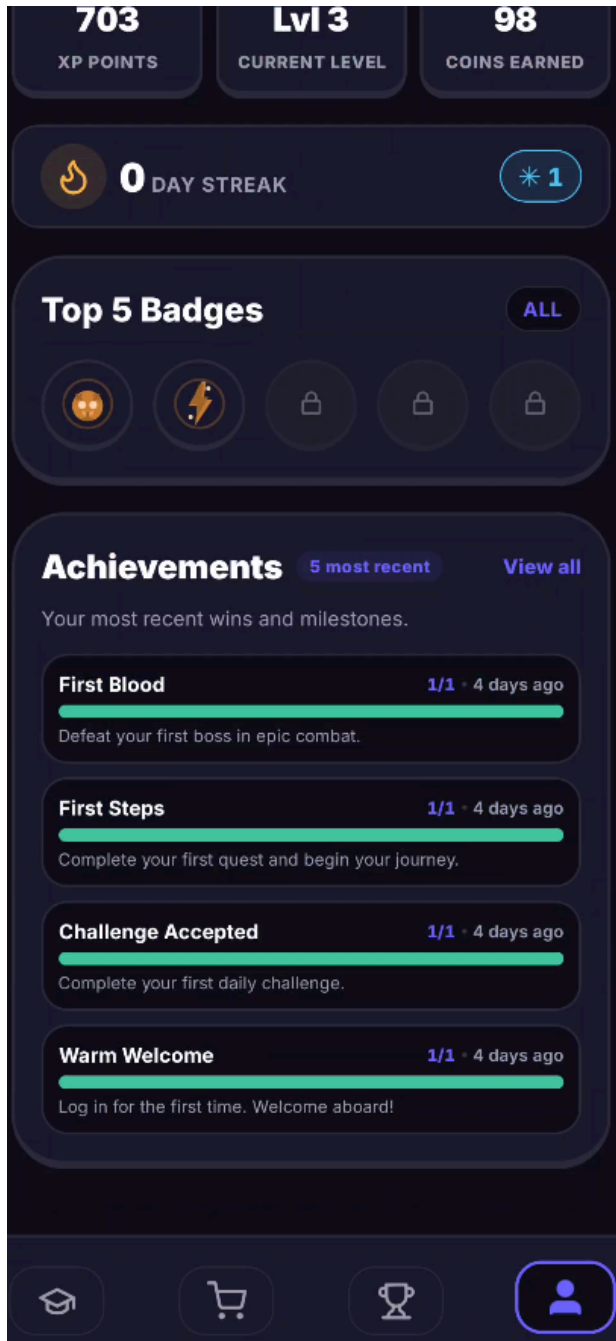
11. In the challenges screen, you will find a “Leaderboards” button where you can see Weekly, Monthly, “Weekly Per-course” Leaderboards of total XP collected.



12. Press on the profile icon in the bottom of the homepage to go to your profile screen where you can edit your avatar and bio.



13. Scroll down in the profile screen to check out your top 5 featured badges (click on “All” to edit your featured badges) and your most 5 recent achievements.



Badges

2 of 20 badges earned · 2/5 featured

Tap an earned badge to feature it on your profile.



Boss Breaker

Defeat powerful bosses to prove your might

1/4



Tough

Defeat 1 boss to cement your dominance.

BRONZE

Featured · tap to remove



Fierce

Defeat 5 bosses to cement your dominance.

SILVER

Locked



Wrecker

Defeat 20 bosses to cement your dominance.

GOLD

Locked



Annihilator

Defeat 75 bosses to cement your dominance.

LEGENDARY

Locked



XP Surge

Accumulate experience points beyond limits

1/4



Sparkling

Earn 500 XP total to reach this milestone.

BRONZE

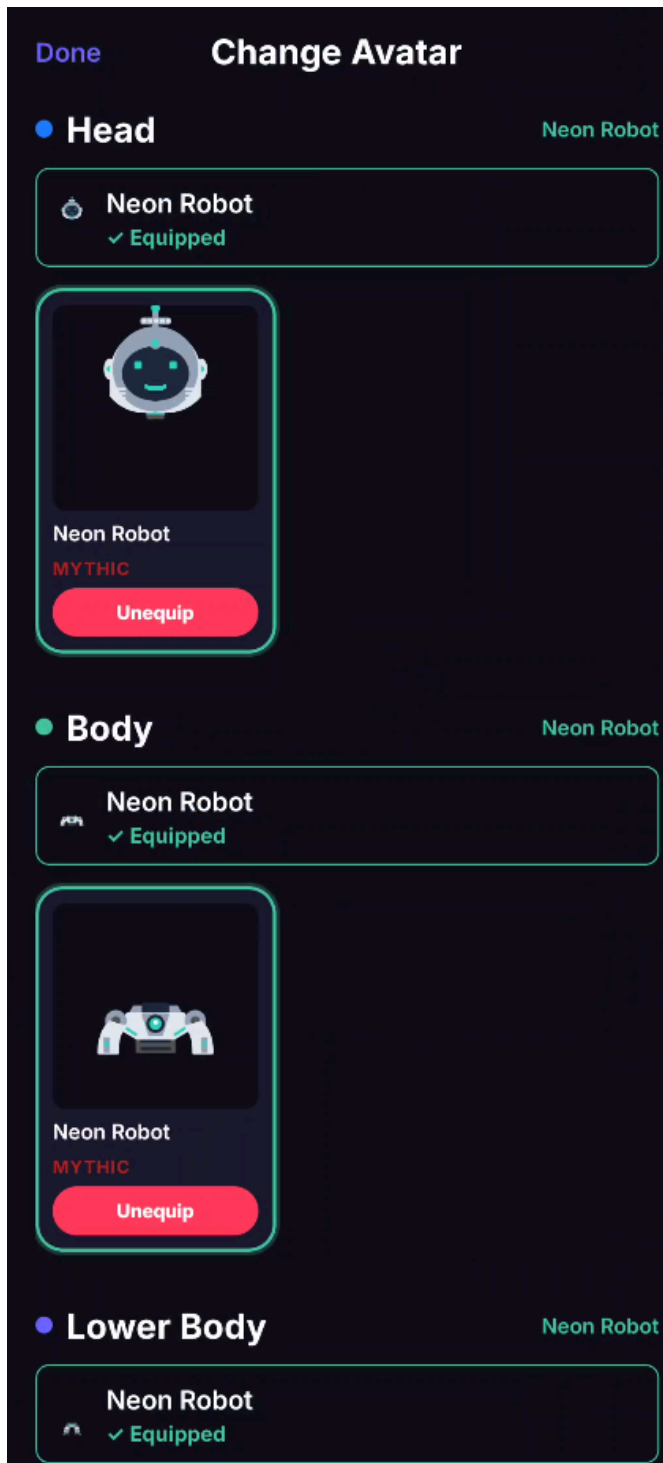
Featured · tap to remove



Glowing

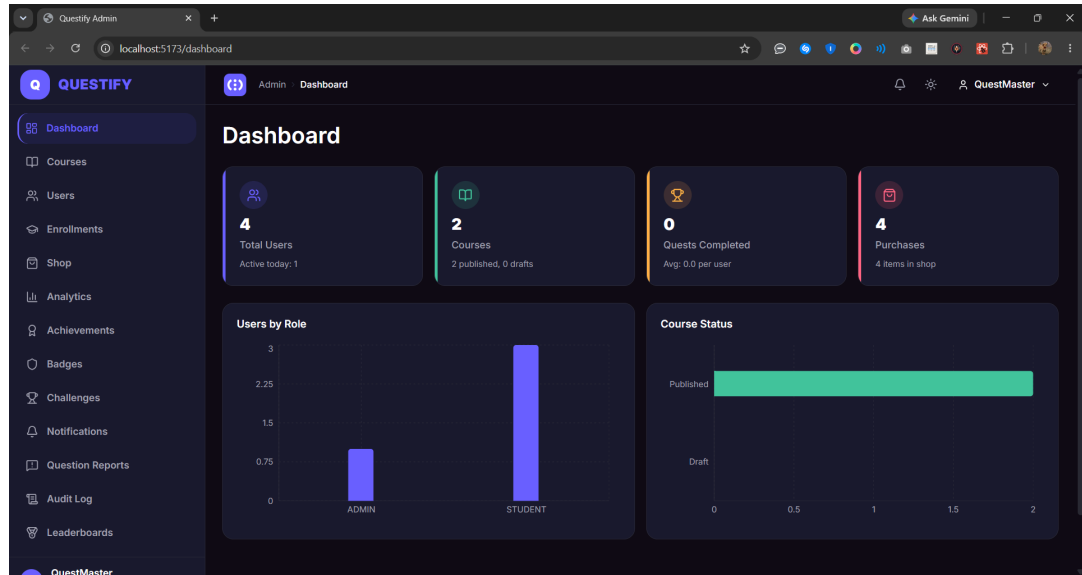
Earn 2,500 XP total to reach this milestone.

14. In profile screen, click on “Change Avatar” to go to inventory and change your equipped items



Admin Dashboard:

1. Open the admin dashboard through localhost.
2. You will be presented with a general analytics dashboard.



3. Press “Courses” to manage course content and stages and to access the AI content generation feature.
4. Each tab (Achievements, Shop, Badges, Challenges, Leaderboards, Question Reports) can be accessed to manage its corresponding module.
5. Press “Audit log” to see all actions taken by other admins for inspection.
6. Press “Notifications” to open the screen where you can broadcast a notification to all users.

5.2 Installation Guide

Prerequisites

Tool	Version	Why	Check
Node.js	22+	Workspace + Turbo	<code>`node -v`</code>
npm	11.6.1 (pinned via <code>`packageManager`</code>)	Repo uses npm workspaces — do not use pnpm/yarn	<code>`npm -v`</code>
PostgreSQL	15+	API database	<code>`psql --version`</code>
Xcode (macOS, iOS)	latest stable	iOS dev build	<code>`xcodebuild -version`</code>
Android Studio + emulator	latest stable	Android dev build	<code>`adb --version`</code>

Install

Shell

```
cd questify
```

```
npm install
```

```
cp apps/api/.env.example apps/api/.env  
cp apps/admin/.env.example apps/admin/.env  
cp apps/mobile/.env.example apps/mobile/.env
```

Environment (minimum required to run)

API — apps/api/.env

Var	Required	Purpose
`DATABASE_URL`	yes	`postgresql://user:password@localhost:5432/questify`
`JWT_SECRET`	yes	Long random string. Generate with `openssl rand -hex 32`
`PORT`	no	Defaults to `8080`
`CORS_ORIGIN`	no	Defaults to `http://localhost:5173` (Admin)

Admin — apps/admin/.env

Var	Required	Purpose
`VITE_API_URL`	yes	`http://localhost:8080` in dev

Mobile — apps/mobile/.env

Var	Required	Purpose
<code>`EXPO_PUBLIC_API_BASE_URL`</code>	yes	API URL as seen from device. <code>`http://localhost:8080/api`</code> for emulator, your LAN IP for a physical device (e.g. <code>`http://192.168.1.4:8080/api`</code>)
<code>`EXPO_PUBLIC_API_DEBUG_LOGGING`</code>	no	Verbose API logs in dev. <code>`true`</code> / <code>`false`</code>

Database setup

```
Shell
npm run build:schemas

npm run generate

npm run migrate

cd apps/api && npx prisma db seed
```

Run

```
Shell
npm run dev
```

Mobile development build

This project uses native modules and cannot run in Expo Go. You must use a development build. Connect a phone to your PC using a USB device to build the app for the first time.

First run

Shell

```
npm run android
```

```
# or: npm run ios
```

Chapter Six:

Conclusion & Future

Work

6.1 Conclusions

Questify is a full-stack, AI-augmented gamified learning platform that unifies instructor-led course creation, mobile learner engagement, and retrieval-augmented assistance. By integrating a multi-stage quest model with a robust gamification engine—including XP, levels, and streak mechanics—the platform successfully fosters long-term learner engagement. The system demonstrates a reliable pipeline for AI-based content generation, grounded in course-specific documents to ensure factual accuracy and curriculum alignment.

6.2 Evaluation Overview

To assess the quality of content produced by the AI Content Generation module, a self-evaluation was conducted on 13 AI-generated quests, comprising 75 individual stages, spanning 4 distinct courses: Software Testing and Quality Assurance, Cloud Computing, Big Data Analysis, and User Interface Design. Each quest was generated by the module directly from its corresponding lecture material and structured into a mix of stage types: Informational (33.3%), Multiple-Choice Question (49.3%), and Boss Battle (17.3%). Each stage was rated on five criteria: correctness, difficulty fit, relevance, clarity, and engagement, along with a categorical hallucination check.

6.2.1 Overall Content Quality

Across all 75 evaluated stages, the module demonstrated strong performance. Correctness scored a mean of 4.95/5 (96% perfect), Difficulty Fit 4.72/5, Relevance 4.39/5, Clarity 4.44/5, and Engagement 4.37/5. Most notably, 0% of

stages were flagged for hallucinated content. At the quest level, 84.6% of generated quests were rated 4/5 or higher, indicating that the majority of content required little to no revision to be considered classroom-ready. Our results align with recent findings in the field; for instance, Mucciaccia et al. [10] reported that their automated system generated 91% valid questions, with their automatic evaluator achieving 89.9% accuracy compared to human assessment, validating the effectiveness of LLM-based question generation frameworks.

6.2.2 Results by Course

Performance was consistent across all four courses. Cloud Computing produced the strongest results (4.75 average quality), while Big Data Analysis performed at 3.67. This suggests the module generalizes well across different domains, though content-heavy lectures may be marginally more challenging for the generation pipeline than concept-based ones.

6.2.3 Results by Stage Type

Boss Battle stages scored highest across nearly every dimension, suggesting the module effectively synthesizes cumulative, high-stakes assessment content. MCQ stages showed the lowest relevance scores, occasionally testing minor details rather than central concepts.

6.2.4 Observed Weaknesses

While overall ratings were high, a small number of stages revealed patterns regarding difficulty calibration and relevance drift, where the module occasionally focused on tangential statistics rather than core concepts. No stage was rated below 3/5 on any metric.

6.2.5 Summary of Key Findings

The system achieved a 0% hallucination rate across 75 stages. 96% of stages achieved a perfect correctness score, and 84.6% of quests were rated Good or Excellent. This confirms the module reliably extracts factual content and indicates strong potential for educational application.

6.3 Future Work

While the current implementation demonstrates that a gamified, AI-assisted learning platform can be delivered as an integrated system of three applications sharing a single validation layer, several directions remain open for both engineering refinement and empirical validation. The work outlined below is grouped into four strands, ordered by their proximity to the present architecture.

- **Lecturer role and stakeholder release:** The admin dashboard already exposes the primitives required for a lecturer persona like course, user, badge, shop, and analytics management but a dedicated **INSTRUCTOR** role with class-cohort scoping, assignment authoring, and per-cohort progress analytics is not yet exposed. Releasing this role to external stakeholders such as **university professors** is a near-term priority, and the validation and authorization layers are already structured to accommodate it with minimal surface-area change.
- **Planned empirical study:** The platform's effect on learning outcomes has not yet been measured quantitatively. A controlled within-subjects study is planned, recruiting approximately **60–90 undergraduate** participants and randomly assigning them to a Questify condition and a

non-gamified control condition delivering equivalent content. Primary outcomes will be learning gain measured by a pre/post knowledge assessment, time-on-task, and one-week delayed retention. Secondary outcomes will be self-reported engagement (an adapted intrinsic motivation inventory) and session frequency over a four-week deployment. Moderating variables to be analyzed include prior GPA, gender, and prior exposure to gamified learning tools. The hypothesis to be tested is that participants in the Questify condition will exhibit significantly higher learning gain and retention than controls.

- **Social and multi-user features:** At present, engagement is mediated exclusively through the individual leaderboard and personal progress feed. A proper social layer is planned in three phases of increasing complexity. The first phase introduces a friendship graph and a friend-scoped activity feed derived from existing leaderboard snapshot deltas and quest-completion events, requiring no new data acquisition. The second phase adds cooperative challenges in which small groups of learners attempt shared boss battles, reusing the existing stage engine with aggregated scoring. The third phase introduces asynchronous and real-time player-versus-player duels, which will require matchmaking, anti-cheat measures, fairness invariants for differing content exposure, and a ranking system.
- **Pedagogical depth and engagement:** The current stage taxonomy — informational, MCQ, and boss battle — is intentionally narrow so that authoring and validation remain tractable. Future iterations will broaden this taxonomy to include timed **rapid-fire drills**, **drag-to-order sequence questions**, **code-input** stages for computing

topics, and branching scenario narratives. Stage authoring will be coupled with an adaptive-difficulty layer that modulates item selection based on rolling per-user accuracy and response time, so that progression is calibrated rather than uniform. The existing animation stack will be extended into a coherent motion language for milestone events — level-up, badge unlock, and boss introductions — replacing the current mostly-static transitions.

References

- [1] Dee, T.S., Jacob, B.A., Simkin, B.A. and Waddington, R.J. (2016) 'Persistence patterns in massive open online courses', *The Journal of Higher Education*, 87(2), pp. 206–242.
- [2] Guettala, M., Bourekkache, S., Kazar, O. and Harous, S. (2024) 'Generative artificial intelligence in education: Advancing adaptive and personalized learning', *Acta Informatica Pragensia*, 2024(3), pp. 460–489.
- [3] Hamari, J., Koivisto, J. and Sarsa, H. (2014) 'Does gamification work? A literature review of empirical studies on gamification', in *Proceedings of the 47th Hawaii International Conference on System Sciences*. IEEE, pp. 3025–3034.
- [4] Kizilcec, R.F., Saltarelli, A.J., Reich, J. and Cohen, G.L. (2020) 'Student engagement and persistence in massive open online courses'.
- [5] Ryan, R.M., Rigby, C.S. and Przybylski, A. (2006) 'The motivational pull of video games: A self-determination theory approach', *Motivation and Emotion*, 30(4), pp. 347–363.
- [6] Schilling, M. et al. (2024) 'Enhancing AI tutoring in robotics education: Evaluating the effect of retrieval augmented generation and fine-tuning on large language models'. University of Münster.
- [7] Wood, W. and Rünger, D. (2016) 'Psychology of habit', *Annual Review of Psychology*, 67, pp. 317–340.
- [8] Pedreira, O., Garcia, F., Piattini, M., Cortiñas, A., Cerdeira-Pena, A. (2020). An architecture for software engineering gamification. *Tsinghua Science and Technology*, 25, 776-797.

- [9] Biancini, G., Ferrato, A., Limongelli, C. (2024). Multiple-Choice Question Generation Using Large Language Models: Methodology and Educator Insights. In Adjunct Proceedings of the 32nd ACM Conference on User Modeling, Adaptation and Personalization (UMAP Adjunct '24).
- [10] Mucciaccia, S. S., Paixao, T. M., Mutz, F., De Souza, A. F., Badue, C. S., Oliveira-Santos, T. (2025). Automatic Multiple-Choice Question Generation and Evaluation Systems Based on LLM: A Study Case With University Resolutions. In Proceedings of the 31st International Conference on Computational Linguistics, pp. 2246-2260.